

Deep Learning & Computer Vision

Day 4: Detailed learning notes

From the calculation inside one neuron to recognising clothes, finding objects, and estimating body positions.

A companion to all three Day 4 notebooks

Neural networks; convolutional networks and transfer learning; computer vision with OpenCV and YOLO.

How to use these notes

Read alongside the relevant notebook. Follow the small calculations first, then run the code and inspect the pictures and training curves. Each practical section explains the purpose of its code, the shapes of the data, and how to interpret the result. Prepared from the supplied 34-slide deck,

speaker notes, and three notebooks, including their saved outputs. Mathematical examples and parameter counts were checked independently. Training and YOLO results are identified as supplied results; the large models were not retrained for these notes.

Contents

1	Starting Day 4	4
1.1	The question for today	4
1.2	Connecting this lesson to Day 3	4
1.3	Reading order and the three notebooks	4
1.4	Setup, runtimes, and result provenance	5
2	One artificial neuron, understood completely	5
2.1	The imports and random seed	5
2.2	Inputs, weights, bias, and output	6
2.3	Exercise 1: changing the circumstances	6
3	Activation functions and why they matter	7
3.1	Why use a nonlinear activation?	7
3.2	Sigmoid	7
3.3	ReLU and tanh	7
3.4	Softmax for several mutually exclusive classes	8
4	Fashion-MNIST: understanding the data before training	10
4.1	What the dataset contains	10
4.2	Downloading and decoding the files	10
4.3	Inspecting examples and class counts	11
4.4	Converting pixel values to floating-point inputs	12
5	Building the first neural network	13
5.1	Layers and tensor shapes	13
5.2	Where the 101,770 parameters come from	13
6	How learning changes the weights	14
6.1	Forward pass and loss	14
6.2	Backpropagation and the chain rule	14
6.3	Gradient descent and learning rate	15
6.4	What Adam does	15
6.5	Epochs, batches, and validation examples	15
7	Reading curves and making predictions	17
7.1	The training-history plotting function	17
7.2	Evaluating on the test set	18
7.3	The probability vector for one image	18
7.4	Reading the confusion matrix	19
7.5	Per-class recall and difficult examples	20
8	Overfitting, dropout, and early stopping	22
8.1	The larger network	22
8.2	What dropout changes during training	23
8.3	Exercise 2: 32 or 512 hidden neurons	24
8.4	Exercise 3: early stopping	25
9	Notebook 2: images, channels, and local structure	25
9.1	Installing and importing the tools	25
9.2	What a colour image contains	26

10 Convolution, worked out by hand	27
10.1 A filter produces one number from a local patch	27
10.2 The complete tiny-image calculation	28
10.3 Applying both filters to the photograph	29
10.4 Exercise 1: blur and sharpening	30
11 Pooling and the complete CNN architecture	30
11.1 Max pooling with a numerical example	30
11.2 Reloading Fashion-MNIST and adding a channel axis	31
11.3 Building the CNN	32
11.4 Counting convolution parameters	32
11.5 Training and reading the result	33
11.6 Checking the hard-coded comparison table	34
12 Looking inside a CNN and extending it	35
12.1 Building a model that stops at an intermediate layer	35
12.2 The filter weights themselves	36
12.3 Exercise 2: add a third convolution	36
13 Transfer learning: starting with learned weights	37
13.1 What is transferred?	37
13.2 The Imagenette experiment	37
13.3 Training the pretrained classifier	38
13.4 Training from initial weights	38
13.5 Reading and comparing the saved results	39
13.6 Choosing images for a visual check	40
13.7 Optional longer run	41
14 Computer vision: understanding the task	42
14.1 Preparing the notebook	42
15 OpenCV: pixels, transformations and drawing	43
15.1 Loading an image and ordering colour channels	43
15.2 Resize, grayscale, blur, edges and crop	44
15.3 Drawing annotations	45
15.4 Exercise 1: rotation and reflection	45
16 Object detection with YOLO	46
16.1 Loading a detector and making predictions	46
16.2 Reading boxes, classes and scores	47
16.3 Timing the complete call	47
16.4 Counting and confidence thresholds	48
16.5 Exercise 2: keep only people	48
17 Classification, masks and body keypoints	49
17.1 Whole-image classification	49
17.2 Instance segmentation	49
17.3 Using the bus mask to make a cutout	49
17.4 Pose estimation	51
17.5 How keypoints can support an application	52
18 Your own photographs, video and homework	52
18.1 Understanding the upload cell	52
18.2 Video and a local webcam	53

18.3 A careful three-photo exercise	53
19 Connecting vision to other deep learning ideas	53
19.1 Sequential data, RNNs and LSTMs	53
19.2 Attention and Transformers	54
19.3 Vision Transformers	54
19.4 Application ideas from the slides	54
20 Review questions with explained answers	55
21 A compact vocabulary reference	56
22 Sources, coverage and result provenance	57
22.1 Primary references for further reading	57

1 Starting Day 4

Material covered: Slides 1–5; introductions and running order in the speaker notes.

1.1 The question for today

A photograph contains a large collection of pixel measurements. Somewhere in those measurements there may be a shoe, a face, or a vehicle. Recognising the object requires combining information across many pixels. A single bright pixel tells us very little on its own.

On earlier days, the input columns already described useful quantities such as age, fare, income, or the shape of a cell nucleus. In image learning, the model often begins much closer to the recorded signal itself. A neural network can learn intermediate representations that help turn those measurements into a prediction.

For example, a network may learn combinations that respond to local edges, textures, and larger arrangements of shapes. These descriptions help us understand what layers can represent. Individual neurons are not guaranteed to acquire neat human labels such as “collar detector” or “cat detector”.

Deep learning refers to learning with neural networks containing multiple processing layers. There is no universal numerical boundary for how many layers make a network deep. Today’s first model has one hidden layer because it gives us a manageable place to understand the calculations. Later examples add convolutional layers and reuse much larger pretrained networks.

1.2 Connecting this lesson to Day 3

The evaluation principles from Day 3 still apply. Training examples adjust a model. Validation examples help select settings and monitor learning. A final test set assesses the chosen procedure after development decisions have been fixed.

A random forest can accept flattened image pixels or carefully designed image features. Learning from raw pixels becomes difficult when objects shift position, vary in size, or appear against different backgrounds. The forest does not automatically reuse a local pattern detector across image positions. Convolutional networks introduce that useful structure into their architecture.

Deep learning also needs deliberate choices about representation, preprocessing, labels, data splits, and evaluation. Learning features does not remove those responsibilities. A useful baseline remains valuable, including the simpler image classifiers you saw on Day 3.

1.3 Reading order and the three notebooks

Notebook	Main ideas	Hands-on block
1: Neural networks	One neuron, activations, Fashion-MNIST, dense network, training curves, dropout	Parts 1–3 at slide 12; Parts 4–5 at slide 16.
2: CNNs and transfer learning	Image channels, filters, pooling, CNN training, pretrained classification	All parts at slide 25.
3: Computer vision	OpenCV, object detection, classification, segmentation, pose, personal images	All parts at slide 32.

The three-hour plan assigns 10 minutes to the recap, 45 to neural networks, 30 to training behaviour,

a 10-minute break, 45 to CNNs, 32 to computer vision, and eight to the closing discussion. The explanations here provide additional detail for independent study.

1.4 Setup, runtimes, and result provenance

A **runtime** is the environment executing your notebook: its Python process, installed packages, memory, and processor. A GPU can perform many numerical operations in parallel, which helps train neural networks. A CPU can run these examples too, with a different runtime cost. GPU availability and actual speed depend on the environment.

Run the cells in order. Restarting a runtime clears variables and fitted models held in memory. Separate notebooks have separate setup steps. Fashion-MNIST, example photographs, YOLO weights, and Imagenette require downloads when they are absent locally.

The supplied notebooks contain saved outputs from executed runs. Some differ from the slide and speaker-note results. For example, the saved dense classifier reaches 87.2% test accuracy, while the slides report 87.1%. These notes distinguish the two records. A saved output shows what was recorded in the uploaded file; it is not a fresh execution performed for this document.

The local environment used to prepare these notes does not contain TensorFlow, PyTorch, or Ultralytics. The large training and prediction runs were therefore not repeated. The neuron arithmetic, softmax, pooling, convolution example, batch counts, and model-parameter calculations were checked separately. The figures are taken from the notebooks' saved outputs unless stated otherwise.

2 One artificial neuron, understood completely

Material covered: Slide 6; Notebook 1, setup and Part 1.

2.1 The imports and random seed

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import keras
from keras import layers
sns.set_theme(style="whitegrid")

keras.utils.set_random_seed(42) # same random start for everyone -> similar results
print("TensorFlow", tf.__version__)
print("GPU available:", len(tf.config.list_physical_devices("GPU")) > 0)
```

NumPy, shortened to **np**, provides numerical arrays. Matplotlib supplies plotting functions through **plt**. Seaborn sets a visual style. TensorFlow is the numerical learning framework used by this notebook. Keras supplies the model-building and training interface, and **layers** gives convenient access to its layer classes.

Keras 3 supports several backends; this notebook is written for its TensorFlow setup. Later, Ultralytics uses PyTorch for the YOLO examples. The same broad ideas of inputs, parameters, loss, and optimisation apply across these libraries.

`set_random_seed(42)` sets random seeds used in the learning environment. Random initial weights and training randomness can influence a result. A fixed seed improves repeatability, but changes

in hardware, software versions, and numerical operations can still change outputs. The printed version and GPU check help record the environment.

2.2 Inputs, weights, bias, and output

An artificial neuron combines input values into one score and then applies a function to that score:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b, \quad a = f(z).$$

The x values are inputs. Each weight w controls an input's contribution. A positive weight increases the score when a positive input increases; a negative weight decreases it. The bias b shifts the score even when all the inputs are zero. The activation function f converts the score into the neuron's output a .

Consider the question “should I go out tonight?” The three inputs represent sunny weather, having money, and having an exam tomorrow. Each is one for yes and zero for no. The notebook uses weights 2, 3, and -4 , and bias -0.5 .

```
# inputs: 1 = yes, 0 = no
x = np.array([1,      # sunny?
              1,      # have money?
              1])     # exam tomorrow?

w = np.array([2, 3, -4]) # weights: exam tomorrow pushes strongly towards "stay home"
b = -0.5                # bias

total = np.dot(w, x) + b # w1*x1 + w2*x2 + w3*x3 + b
print("Weighted sum =", total)
print("Decision:", "GO OUT " if total > 0 else "STAY HOME ")
```

`np.array` makes numerical vectors. `np.dot(w, x)` multiplies corresponding entries and adds the products. For the supplied inputs,

$$z = 2(1) + 3(1) - 4(1) - 0.5 = 0.5.$$

The decision rule prints “go out” if the score is above zero. Otherwise it prints “stay home”. The Python expression with `if` and `else` chooses between those two strings.

The signs matter: the exam weight and bias are negative in the notebook. Use the calculation above if the slide's small text or extracted text makes a minus sign difficult to see.

Here all inputs use the same zero/one scale, so the exam input has the largest possible single contribution in magnitude. In a real model with differently scaled features, comparing weight magnitudes alone may be misleading.

The weights in this toy example were chosen by a person to make the arithmetic clear. During supervised training, an optimiser adjusts a network's parameters using examples and their known answers.

2.3 Exercise 1: changing the circumstances

```
# Exercise 1
w2 = np.array([2, 3, -6])
print("Exam weight -6:", np.dot(w2, x) + b, "-> stay home")
print("Not sunny      :", np.dot(w, np.array([0, 1, 1])) + b, "-> stay home")
```

With exam weight -6 , the score becomes $2 + 3 - 6 - 0.5 = -1.5$, so the decision is stay home. With the original exam weight -4 and no sunshine, it becomes $0 + 3 - 4 - 0.5 = -1.5$, also stay home.

The solution tests these as two separate changes. If both changes happen together, the score is $0 + 3 - 6 - 0.5 = -3.5$. Setting an input to zero removes that input's weighted contribution. Changing the bias shifts the decision threshold for every possible input combination.

3 Activation functions and why they matter

Material covered: Slide 7; Notebook 1, activation cells.

3.1 Why use a nonlinear activation?

Suppose one layer calculates $u = ax + b$, and the next calculates $v = cu + d$. Substituting the first into the second gives

$$v = c(ax + b) + d = (ca)x + (cb + d).$$

The stacked calculation is still one affine rule in x . The same idea holds for matrices. Nonlinear activations allow the combined network to represent relationships that repeated affine transformations alone cannot create.

An activation is a numerical rule. Its choice affects what a layer can represent and how gradients flow during learning. Hidden layers and output layers can use different activations because they have different jobs.

3.2 Sigmoid

The sigmoid is

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The number e is approximately 2.718. For $z = 0$, the result is $1/(1 + 1) = 0.5$. For a large positive score, the output approaches one. For a large negative score, it approaches zero.

For the neuron's score of 0.5, the output is approximately 0.622. In a fitted binary classifier, the sigmoid output can be interpreted as an estimated class-one probability. Our go-out example has hand-chosen weights and no calibration evidence. Its 0.622 is therefore an illustrative numerical output, and the printed "62% sure" should be read in that limited sense.

Sigmoid becomes very flat at large positive or negative inputs. Its gradient is then small, which can make learning through many such layers difficult. This is one reason other activations are common in hidden layers.

3.3 ReLU and tanh

The rectified linear unit, ReLU, is

$$\text{ReLU}(z) = \max(0, z).$$

Inputs -3 , 0 , and 2.5 produce 0 , 0 , and 2.5 . The function is inexpensive and creates a nonlinear bend at zero. ReLU is a common hidden-layer choice and is used throughout today's small image networks.

For positive inputs its derivative is one, and for negative inputs it is zero. A unit that stays on the negative side for all relevant examples may receive no useful gradient through that activation. Other activations can be considered when this becomes a problem.

The hyperbolic tangent, **tanh**, maps values into $(-1, 1)$ and passes through zero. It appears in the notebook's third plotted panel. It is useful to recognise even though the dense and convolutional examples use ReLU. Like sigmoid, it becomes flat at its extremes.

```
z = np.linspace(-6, 6, 200)
sigmoid = 1 / (1 + np.exp(-z))
relu = np.maximum(0, z)
tanh = np.tanh(z)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, y, name in zip(axes, [sigmoid, relu, tanh], ["Sigmoid", "ReLU", "Tanh"]):
    ax.plot(z, y, linewidth=3)
    ax.axhline(0, color="grey", lw=1); ax.axvline(0, color="grey", lw=1)
    ax.set_title(name, fontsize=16); ax.set_xlabel("weighted sum z")
plt.tight_layout(); plt.show()

print("Sigmoid of our 0.5 total:", round(1 / (1 + np.exp(-0.5)), 3), "-> 62% sure we go out")
```

`np.linspace(-6, 6, 200)` produces 200 evenly spaced inputs. NumPy applies the activation calculations to every entry. `np.maximum(0, z)` compares zero with each score separately. The loop pairs each axis with an output array and function name.

The horizontal and vertical reference lines mark zero. `tight_layout` adjusts spacing between panels, and `show` displays the figure. The activation chart is a picture of the functions themselves, so it does not require training data.

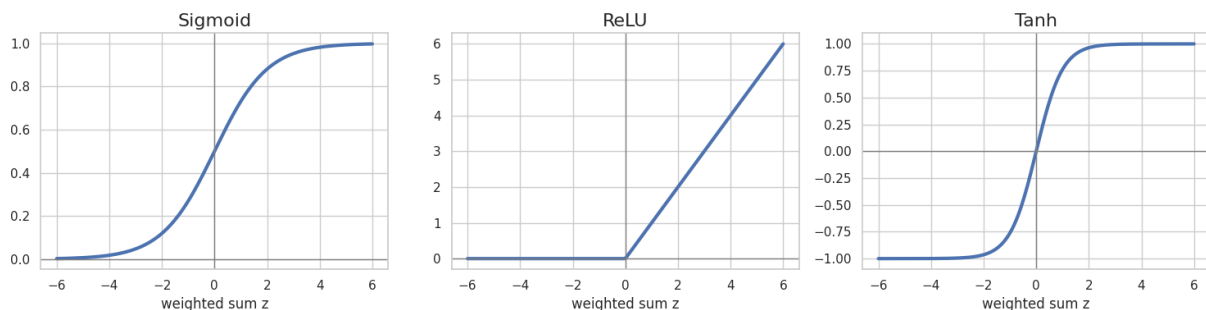


Figure 1: Sigmoid, ReLU, and tanh from the saved notebook output.

3.4 Softmax for several mutually exclusive classes

For scores $z_1, \dots, z_C$, softmax returns

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}.$$

C is the number of classes. Each output is positive, and the outputs sum to one. A higher input score receives a higher output probability.

```
# Softmax: turn raw scores into probabilities
scores = np.array([2.0, 1.0, 0.1])
probs = np.exp(scores) / np.exp(scores).sum()
print("scores      :", scores)
print("probabilities:", probs.round(3), " sum =", probs.sum())
```

For scores $[2, 1, 0.1]$, the exponentials are approximately $[7.389, 2.718, 1.105]$. Their sum is about 11.212. Dividing each entry by that sum gives $[0.659, 0.242, 0.099]$. The first class is the predicted category because its probability is largest.

Softmax compares the scores with one another. Increasing one class score can reduce the probabilities assigned to other classes even if their scores do not change. It suits today's task because each clothing image has one target category.

For numerical stability with large scores, subtract the largest score before taking exponentials:

```
stable = scores - scores.max()
probs = np.exp(stable) / np.exp(stable).sum()
```

This produces the same mathematical probabilities while reducing overflow risk. Keras handles stable calculations internally. Probability normalisation alone does not establish that the model is well calibrated or that it can recognise an unfamiliar class.

4 Fashion-MNIST: understanding the data before training

Material covered: Slides 8, 11 and 18; Notebook 1, Part 2.

4.1 What the dataset contains

Fashion-MNIST contains 70,000 small greyscale clothing images with ten target categories. The standard split contains 60,000 training images and 10,000 test images. Each image has 28 rows and 28 columns, giving 784 pixel values.

Code	Category	Code	Category
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

The notebook shortens T-shirt/top to T-shirt. The label codes identify categories; a larger code does not mean a larger or more valuable item. Each standard training class contains 6,000 images and each test class contains 1,000.

4.2 Downloading and decoding the files

```
import gzip, urllib.request

BASE = "https://raw.githubusercontent.com/zalando-research/fashion-mnist/master/data/fashion/"

def load_fashion(kind):
    """Download one half (train or t10k) of Fashion-MNIST straight from the official GitHub repo
    """
    raw_imgs = urllib.request.urlopen(BASE + f"{kind}-images-idx3-ubyte.gz").read()
    raw_labs = urllib.request.urlopen(BASE + f"{kind}-labels-idx1-ubyte.gz").read()
    imgs = np.frombuffer(gzip.decompress(raw_imgs), np.uint8, offset=16).reshape(-1, 28, 28)
    labs = np.frombuffer(gzip.decompress(raw_labs), np.uint8, offset=8)
    return imgs, labs

X_train_raw, y_train = load_fashion("train")
X_test_raw, y_test = load_fashion("t10k")

class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

print("Training images:", X_train_raw.shape) # (60000, 28, 28)
print("Test images:    ", X_test_raw.shape)  # (10000, 28, 28)
```

The dataset is stored as compressed binary files in the official Fashion-MNIST repository. `urllib.request.urlopen` opens a URL; `read()` obtains its bytes. `gzip.decompress` removes the gzip compression.

The files use the IDX format. The image file begins with a 16-byte header describing the data; the label file has an eight-byte header. The offsets skip those headers. `np.frombuffer` interprets the remaining bytes as unsigned eight-bit integers, written `np.uint8`. Each value can range from zero to 255.

`reshape(-1, 28, 28)` groups the pixel stream into images. The `-1` asks NumPy to infer how many images fit the remaining dimensions. `load_fashion("train")` loads the training pair; `load_fashion("t10k")` loads the test pair. The function returns the image array and its labels together.

The training-image shape `(60000, 28, 28)` means 60,000 images, each with 28 rows and 28 columns. The training labels form a one-dimensional array of 60,000 integers. Correct row alignment is essential: image i must be paired with label i .

4.3 Inspecting examples and class counts

```
# How many of each class? (perfectly balanced)
print(np.bincount(y_train))

plt.figure(figsize=(12, 5))
for i in range(18):
    plt.subplot(3, 6, i + 1)
    plt.imshow(X_train_raw[i], cmap="gray_r")
    plt.title(class_names[y_train[i]], fontsize=11)
    plt.axis("off")
plt.suptitle("First 18 training images", fontsize=15)
plt.tight_layout(); plt.show()
```

`np.bincount(y_train)` counts labels zero through nine. The ten values are all 6,000. The plotting loop creates a three-by-six grid of the first 18 examples. Matplotlib subplot positions begin at one, so the code uses `i + 1`.

`class_names[y_train[i]]` translates a numerical label into a title. `gray_r` is a reversed greyscale colour map, so the plotted background appears light and the clothing dark. The underlying pixel encoding still uses zero for black and 255 for white under conventional greyscale display; the colour map changes how those values are shown.



Figure 2: The first 18 Fashion-MNIST training examples in the saved notebook.

```
print("Label:", class_names[y_train[0]])
print("Shape:", X_train_raw[0].shape, "=", 28 * 28, "numbers")
print(X_train_raw[0][5:15, 5:15]) # a 10 x 10 window of pixels
```

The first training label is ankle boot. The slice `[5:15, 5:15]` selects rows five through fourteen and columns five through fourteen, using zero-based indexing. It produces a ten-by-ten interior window. The notebook's introductory description of a top-left corner is imprecise.

Zero values largely represent the background in this example; higher values mark brighter parts of the recorded object. The printed window contains values up to 244. A whole image is a grid of these measurements, and the learning model receives their numerical values.

4.4 Converting pixel values to floating-point inputs

```
X_train = X_train_raw.astype("float32") / 255.0
X_test  = X_test_raw.astype("float32") / 255.0
print("Min:", X_train.min(), " Max:", X_train.max())
```

`astype("float32")` converts the byte values into 32-bit floating-point numbers. Division by 255 maps the original range to `[0, 1]`: zero stays zero, 255 becomes one, and 128 becomes approximately 0.502.

A consistent numerical scale helps optimisation. It does not make the task easier by revealing the labels. Division by a fixed known constant also avoids learning a statistic from the test set. Apply the same transformation to training and test images.

The raw arrays are retained for inspection while `X_train` and `X_test` hold the scaled inputs. Scaling changes values and storage type; the shapes remain `(60000, 28, 28)` and `(10000, 28, 28)`.

5 Building the first neural network

Material covered: Slides 8 and 11; Notebook 1, Part 3.

5.1 Layers and tensor shapes

A **tensor** is a numerical array with any number of dimensions. A vector has one dimension, a matrix has two, and a batch of greyscale images here has three. The word describes the array structure; it does not imply that every dimension is a spatial direction.

A layer transforms one representation into another. An input layer describes the expected shape. A hidden layer produces intermediate features. The output layer produces values used for the final task.

```
keras.utils.set_random_seed(42)

model = keras.Sequential([
    keras.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax"),
])
model.summary()
```

`Sequential` arranges the layers in order. `Input(shape=(28, 28))` describes one image; the batch dimension is supplied separately. `Flatten` rearranges each image into a vector of 784 values without changing the values or learning any parameters.

`Dense(128, activation="relu")` creates 128 neurons, each connected to all 784 input values. Each neuron calculates its own weighted sum and bias, then applies ReLU. The layer produces 128 activations per image.

The output layer has ten neurons, one for each clothing class. It transforms the 128 hidden activations into ten scores and applies softmax. The predicted category is the position of the largest probability.

A dense model still receives a distinct value for every pixel position after flattening. Its architecture simply does not explicitly exploit nearby positions and shared local filters. It can learn combinations involving nearby pixels, but has to learn that structure through its weights.

Stage	Shape for a batch of B images	Learned parameters
Input	$(B, 28, 28)$	0
Flatten	$(B, 784)$	0
Dense hidden layer	$(B, 128)$	100,480
Dense output layer	$(B, 10)$	1,290

In a model summary, `None` in the first dimension means the batch size is flexible. It does not mean the data is missing.

5.2 Where the 101,770 parameters come from

For a dense layer with d inputs and h neurons, each neuron has d connection weights and one bias. The count is

$$(d + 1)h = dh + h.$$

The first dense layer has $784 \times 128 = 100,352$ connection weights and 128 biases, giving 100,480 parameters. The output layer has $128 \times 10 = 1,280$ weights and ten biases, giving 1,290. The total is

$$100,480 + 1,290 = 101,770.$$

The slides use “weights” informally for this total. **Parameters** is the precise term when biases are included. At four bytes per float32 value, the parameter arrays alone require 407,080 bytes, approximately 398 KiB. Training also stores activations, gradients, and optimiser state, so total training memory is larger.

6 How learning changes the weights

Material covered: Slides 9–10; Notebook 1, compile and fit cells.

6.1 Forward pass and loss

The **forward pass** applies the current layers to the inputs. Initially the weights are random, so the probabilities may give poor answers. We measure those answers using a loss function.

For an example whose true class is y , categorical cross-entropy with a softmax output is

$$L = -\log(p_y),$$

where p_y is the probability assigned to the correct class and the logarithm is natural. If the correct class receives 0.9, the loss is about 0.105. At 0.5 it is about 0.693; at 0.1 it is about 2.303. Confidently assigning a very small probability to the true answer gives a large penalty.

Accuracy uses only the selected category. Loss also responds to the distribution of probabilities. Two models can therefore have the same accuracy and different loss values. A model may become more confident in its wrong predictions while its number of mistakes stays similar, causing validation loss to rise.

6.2 Backpropagation and the chain rule

A **gradient** measures how a small change in a parameter affects the loss locally. Backpropagation efficiently calculates those gradients through the layers, working backward from the loss. It uses the chain rule to connect derivatives through a sequence of calculations.

For a small worked example, take one input $x = 2$, one weight $w = 0.5$, bias $b = 0$, prediction $\hat{y} = wx + b = 1$, and target $y = 3$. Use the illustrative squared loss

$$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(1 - 3)^2 = 2.$$

The derivative with respect to the prediction is $\hat{y} - y = -2$. The prediction changes by $x = 2$ for a unit change in w , so the chain rule gives

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x = -4, \quad \frac{\partial L}{\partial b} = \hat{y} - y = -2.$$

The symbol ∂ indicates a derivative with respect to one variable while considering the others fixed. The negative weight gradient means that a small increase in this weight would reduce the current loss.

6.3 Gradient descent and learning rate

Gradient descent updates a parameter by subtracting the learning rate times its gradient:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}.$$

With learning rate $\eta = 0.1$, the example becomes $w = 0.5 - 0.1(-4) = 0.9$ and $b = 0 - 0.1(-2) = 0.2$. The new prediction is $0.9(2) + 0.2 = 2$, and the loss is now $\frac{1}{2}(2 - 3)^2 = 0.5$.

This small example uses a linear prediction and squared loss to make the arithmetic visible. The clothing model uses many parameters, ReLU, softmax, and cross-entropy. Automatic differentiation calculates its much larger collection of derivatives.

Backpropagation computes gradients. The **optimiser** uses them to update parameters. Keeping these two roles clear makes the training loop easier to follow. A large learning rate can overshoot a useful region; a very small rate can make progress slow. With mini-batches and a nonlinear objective, a particular update is not guaranteed to reduce the full-dataset loss.

6.4 What Adam does

Adam maintains running summaries of recent gradients and their squared magnitudes. These summaries help adapt updates for different parameters and smooth some of the variability in mini-batch gradients. It still uses a learning-rate setting and can still train poorly if the overall setup is unsuitable.

You do not need to manually calculate Adam's internal arrays to use the notebook. You should know that it is the rule responsible for updating the weights, while the loss tells the procedure what improvement means.

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

`compile` configures the learning process. `optimizer="adam"` selects the optimiser. The loss name `sparse_categorical_crossentropy` matches integer category labels such as zero, six, or nine. Here “sparse” describes the label representation. It does not mean there are few images or mostly empty input pixels.

If targets were ten-entry one-hot vectors, categorical cross-entropy would use that different representation. `metrics=["accuracy"]` asks Keras to report accuracy alongside the optimised loss. Compiling a model configures it; the training examples have not yet adjusted its weights.

6.5 Epochs, batches, and validation examples

An **epoch** is one pass through the examples actually used for training. A **batch** is a group processed for one update in this standard training loop. After calculating a batch loss and gradients, the optimiser updates the parameters, then continues with the next batch.

```
history = model.fit(X_train, y_train,
                   epochs=10,
                   batch_size=32,
                   validation_split=0.1)
```

The array contains 60,000 images. `validation_split=0.1` reserves 6,000, leaving 54,000 for parameter updates. With NumPy arrays, Keras takes this validation portion from the end of the supplied arrays before shuffling. This option does not automatically stratify the validation subset.

Batch size 32 gives

$$\left\lceil \frac{54,000}{32} \right\rceil = 1,688$$

updates per epoch, where the ceiling means round up. There are 1,687 full batches of 32 and a final batch of 16. Ten epochs therefore produce 16,880 updates, with validation evaluation between epochs. The same 6,000 validation examples remain reserved throughout this fit.

The default training shuffle changes the order of the training examples between epochs. It does not move validation examples back into training. Validation data provides feedback for human choices and callbacks, even though ordinary gradient updates do not directly use it.

The returned `History` object stores one set of summary values per epoch. `history.history` is a dictionary containing lists such as training accuracy, validation accuracy, training loss, and validation loss.

7 Reading curves and making predictions

Material covered: Slides 13 and 15; Notebook 1, end of Part 3 and Part 4.

7.1 The training-history plotting function

```
def plot_history(h, title=""):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
    axes[0].plot(h.history["accuracy"], "o-", label="training")
    axes[0].plot(h.history["val_accuracy"], "o-", label="validation")
    axes[0].set_title("Accuracy " + title, fontsize=14); axes[0].set_xlabel("epoch"); axes[0].legend()
    axes[1].plot(h.history["loss"], "o-", label="training")
    axes[1].plot(h.history["val_loss"], "o-", label="validation")
    axes[1].set_title("Loss " + title, fontsize=14); axes[1].set_xlabel("epoch"); axes[1].legend()
    plt.tight_layout(); plt.show()

plot_history(history)
```

The function receives a Keras History object. Its first panel plots training and validation accuracy; the second plots the corresponding losses. The string "o-" requests markers connected by lines. Legends identify the two datasets.

The notebook plots list positions without supplying explicit epoch numbers. Its horizontal labels therefore begin at zero: plotted position zero corresponds to the first completed epoch, and position nine to the tenth. If you want labels one through ten, pass `range(1, len(values) + 1)` as the horizontal coordinates.

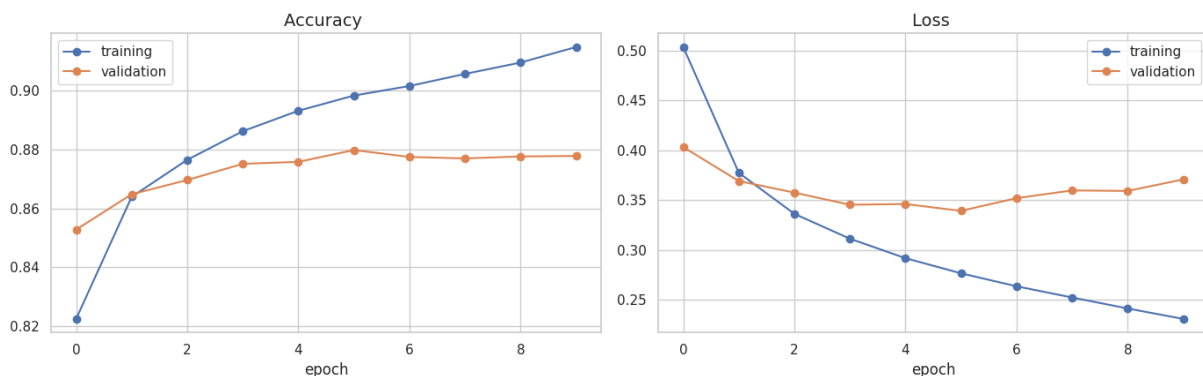


Figure 3: Saved learning curves for the first dense model. Accuracy and loss tell related but different stories.

The saved training accuracy rises from about 82.3% to 91.5%. Validation accuracy rises from about 85.3% to 87.8%. Validation loss reaches approximately 0.339 around epoch six, then ends around 0.371, while training loss continues falling.

A sustained improvement in both training and validation accuracy suggests useful learning. A growing gap together with worsening validation loss suggests overfitting. One upward fluctuation in validation loss can also be sampling or optimisation noise, so inspect the pattern across epochs.

Training accuracy is accumulated while weights are changing during the epoch. Validation is measured afterward with the current fixed weights. This can help explain why validation accuracy is sometimes above the reported training accuracy early in training. Dropout adds another difference

in the later experiments.

7.2 Evaluating on the test set

```
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.1%}    (random guessing = 10%)")
```

`evaluate` computes loss and the requested metrics without ordinary training updates. The return order follows the model's configuration: here test loss and test accuracy. `verbose=0` suppresses the progress display.

The saved notebook reports 87.2% test accuracy. The slides and speaker notes report 87.1% for their run. Both are examples of this architecture's performance under the supplied setup. Their difference is not evidence by itself that either run is broken.

Uniform random guessing over ten classes has expected accuracy 10%. That is a reference expectation, not a promise that a finite random set of guesses lands exactly on 10%.

The exercises evaluate several architectures on this same test set. If you use those test results to select your model, the test set has influenced development. Use validation results for choices and preserve a separate final evaluation for a formal performance claim.

7.3 The probability vector for one image

```
probs = model.predict(X_test[:12], verbose=0)
print("Probabilities for the first test image:")
for name, p in zip(class_names, probs[0]):
    print(f" {name:11s} {p:.3f}")

preds = probs.argmax(axis=1)
plt.figure(figsize=(12, 5))
for i in range(12):
    plt.subplot(2, 6, i + 1)
    plt.imshow(X_test[i], cmap="gray_r"); plt.axis("off")
    ok = preds[i] == y_test[i]
    plt.title(f"{class_names[preds[i]]}\n({probs[i].max():.0%})", color="green" if ok else "red",
              fontsize=11)
plt.suptitle("Predictions (green = right, red = wrong)", fontsize=14)
plt.tight_layout(); plt.show()
```

`X_test[:12]` selects the first twelve images. The predicted output has shape (12,10): twelve rows with ten class probabilities each. `probs[0]` is the first image's vector. The loop prints it next to the class names.

`argmax(axis=1)` finds the largest entry across the ten classes in each row. The result is a twelve-element array of predicted category codes. In the display loop, `ok` checks whether prediction and known target match, then selects a green or red title.

The saved first-image output gives about 0.992 to ankle boot and 0.008 to sneaker. The speaker notes describe an earlier run with 0.964 and 0.036. Printed probabilities rounded to zero are not necessarily exactly zero. Similarly, a title rounded to 100% should not be interpreted as certainty.

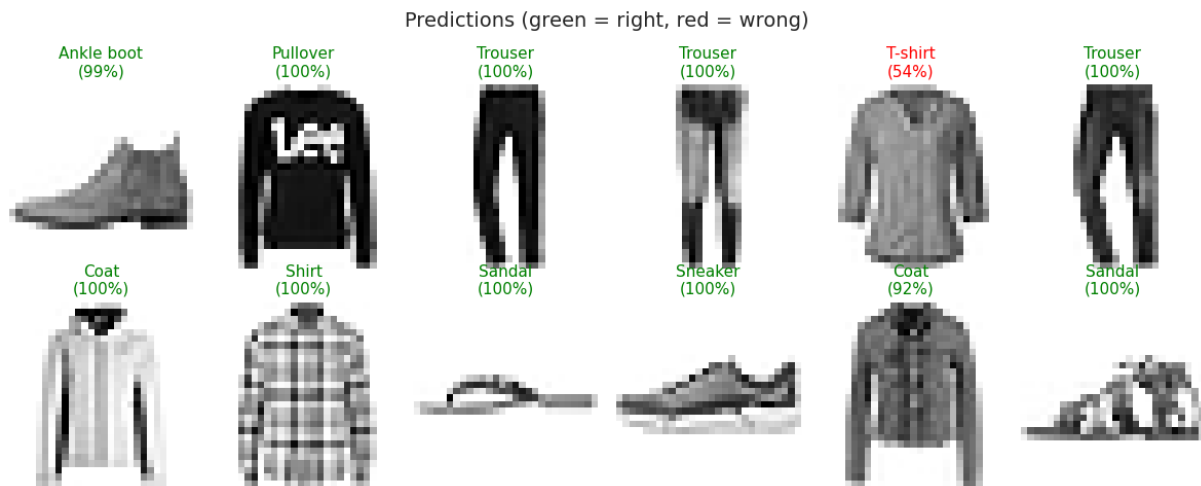


Figure 4: Predictions saved in Notebook 1. Red text marks a mismatch with the known label.

A softmax classifier allocates probability across its available categories. It has no automatically learned “unknown object” output. Giving it an unrelated picture can still produce a confident clothing prediction.

7.4 Reading the confusion matrix

```
from sklearn.metrics import confusion_matrix, classification_report

y_pred = model.predict(X_test, verbose=0).argmax(axis=1)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted", fontsize=13); plt.ylabel("True", fontsize=13)
plt.title("Confusion matrix - dense network", fontsize=15)
plt.show()
```

`model.predict(X_test)` produces probabilities for every test image, and `argmax` selects their categories. `confusion_matrix` counts true/predicted label pairs. Rows are actual classes; columns are predicted classes. The diagonal contains correct predictions.

`annot=True` writes counts inside the heatmap. `fmt="d"` prints them as integers. The class-name lists make numerical row and column positions readable. `classification_report` is imported in this cell but is not called in the supplied notebook.

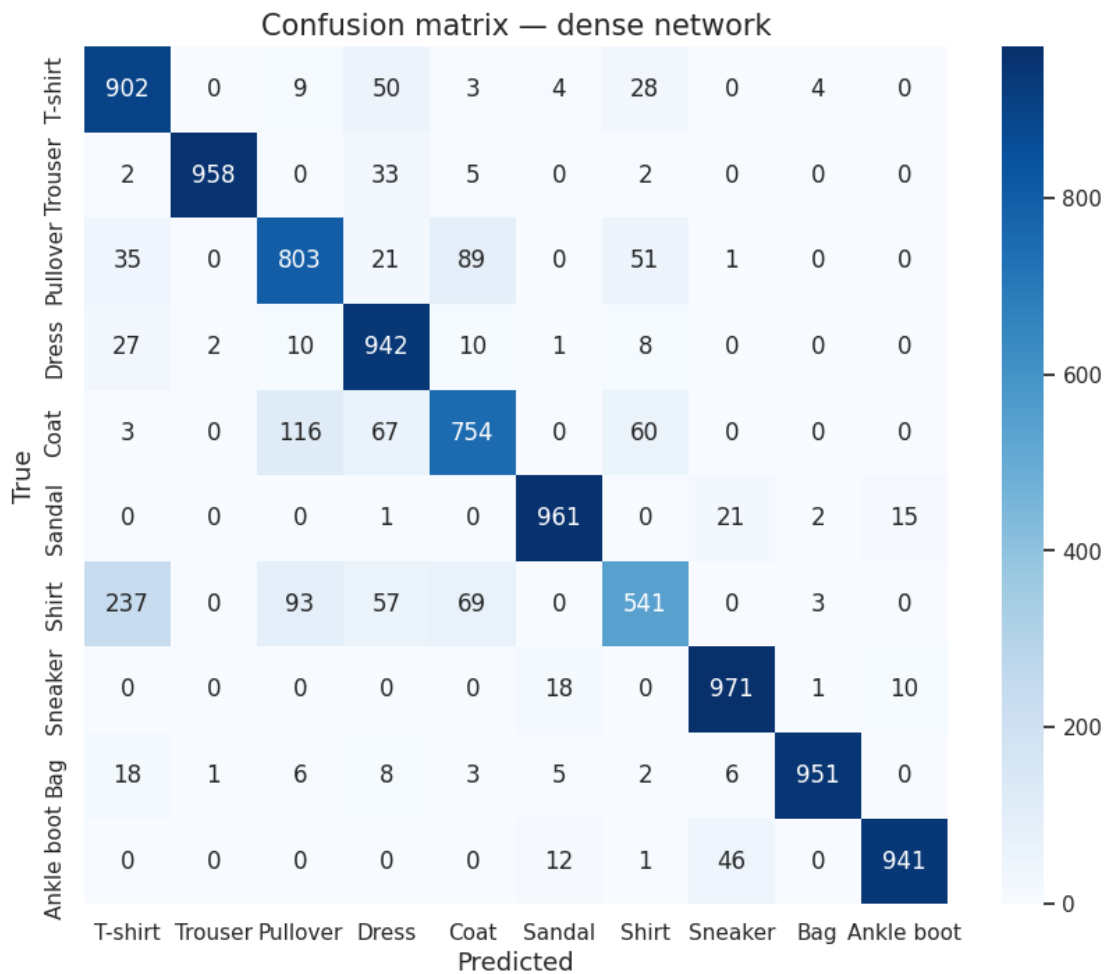


Figure 5: The saved dense-network confusion matrix. Each true class has 1,000 test examples.

The saved shirt row contains 541 correct shirt predictions and 237 T-shirt predictions, along with other errors. The slide's earlier run reports 538 correct and 249 shirts called T-shirts. Keep the run labels attached to these counts when explaining them.

7.5 Per-class recall and difficult examples

```
per_class = cm.diagonal() / cm.sum(axis=1)
order = np.argsort(per_class)[::-1]
for i in order:
    print(f"{class_names[i]:11s} {per_class[i]:.1%}")
```

`cm.diagonal()` retrieves the correct count for every class. `cm.sum(axis=1)` counts the actual examples in each row. Dividing gives **recall for each class**, which the teaching materials informally call per-class accuracy.

For shirts in the saved matrix, recall is $541/1000 = 54.1\%$. Precision for shirt would instead divide correct shirt predictions by the whole predicted-shirt column. These two denominators answer different questions.

`np.argsort` returns positions in ascending score order; `[::-1]` reverses them so the strongest class appears first. The saved recalls are sneaker 97.1%, sandal 96.1%, trouser 95.8%, bag 95.1%, dress 94.2%, ankle boot 94.1%, T-shirt 90.2%, pullover 80.3%, coat 75.4%, and shirt 54.1%.

```
# Show some shirts the network got wrong
wrong = np.where((y_test == 6) & (y_pred != 6))[0][:12]
plt.figure(figsize=(12, 5))
for k, i in enumerate(wrong):
    plt.subplot(2, 6, k + 1)
    plt.imshow(X_test[i], cmap="gray_r"); plt.axis("off")
    plt.title("said: " + class_names[y_pred[i]], fontsize=11, color="red")
plt.suptitle("Real shirts the network got wrong", fontsize=14)
plt.tight_layout(); plt.show()
```

The mask `(y_test == 6) & (y_pred != 6)` selects actual shirts that received another prediction. Parentheses group the comparisons, and `&` combines them element by element. `np.where(...)[0]` retrieves matching positions; `[:12]` selects the first twelve.



Figure 6: Saved examples of misclassified shirts. The title gives the predicted category.

At this resolution, collars, sleeves, and texture can be hard to distinguish. Examining errors helps identify ambiguity, limited representation, and possible label issues. The twelve displayed examples illustrate failure cases; they are not a random estimate of the error rate.

8 Overfitting, dropout, and early stopping

Material covered: Slides 14 and 16; Notebook 1, Part 5 and exercises.

8.1 The larger network

```
def big_model(dropout=0.0):
    keras.utils.set_random_seed(42)
    m = keras.Sequential([keras.Input(shape=(28, 28)), layers.Flatten()])
    for _ in range(2):
        m.add(layers.Dense(512, activation="relu"))
        if dropout > 0:
            m.add(layers.Dropout(dropout)) # switch off a random 30% of neurons at every
            training step
    m.add(layers.Dense(10, activation="softmax"))
    m.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return m

big = big_model(dropout=0.0)
h_big = big.fit(X_train, y_train, epochs=30, batch_size=128, validation_split=0.1, verbose=0)
print("No dropout -> train acc %.1f%% val acc %.1f%%" % (100 * h_big.history["accuracy"][-1],
    100 * h_big.history["val_accuracy"][-1]))
plot_history(h_big, "(no dropout)")
```

`big_model` constructs a new model whenever it is called. It begins with the same image shape and flattening, then loops twice to add two dense layers of 512 ReLU neurons. The underscore in `for _ in range(2)` is a conventional variable name when the loop counter is not otherwise needed.

The conditional adds dropout after each hidden layer only when the rate is above zero. The output still has ten softmax units. The function compiles the model and returns it. Resetting the seed inside the function controls its initial randomness.

The parameter count is

$$(784 + 1)512 + (512 + 1)512 + (512 + 1)10 = 669,706.$$

That includes 401,920 parameters in the first hidden layer, 262,656 in the second, and 5,130 in the output. The architecture has much more capacity than the first 101,770-parameter model.

The fit uses 30 epochs and batches of 128, so it performs $\lceil 54000/128 \rceil = 422$ updates per epoch. `verbose=0` hides progress during these longer fits. Reading `[-1]` from a history list retrieves the last epoch's value.

The saved run without dropout ends at 96.7% training accuracy and 88.8% validation accuracy. Its validation loss rises substantially during later epochs as training loss keeps falling. The model is learning details that improve the training fit without a corresponding validation improvement.

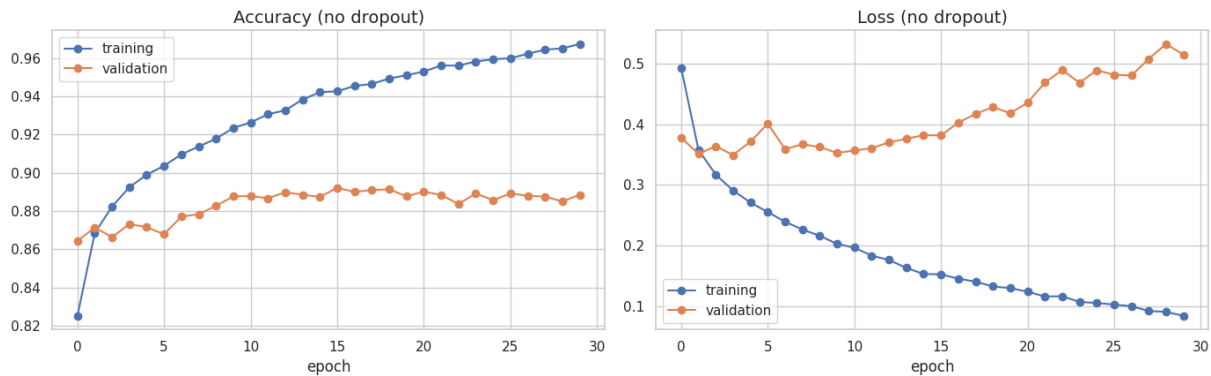


Figure 7: Saved training curves for the larger model without dropout.

8.2 What dropout changes during training

Dropout randomly sets a fraction of activations to zero during training. For rate 0.3, each relevant activation has probability 0.3 of being dropped under the default mask behaviour. The sampled fraction in a particular small group need not be exactly 30%.

The remaining activations are scaled by $1/(1 - 0.3) = 1/0.7$ during training. This keeps their expected contribution consistent. For example, an activation of 2 is zero with probability 0.3 and about 2.857 with probability 0.7, giving expected value two.

At evaluation and prediction time, dropout is disabled and the layer passes activations through. It does not permanently remove neurons or their learned weights. The random disruption can reduce reliance on particular combinations of hidden activations, helping regularise learning.

```
drop = big_model(dropout=0.3)
h_drop = drop.fit(X_train, y_train, epochs=30, batch_size=128, validation_split=0.1, verbose=0)
print("With dropout -> train acc %.1f%% val acc %.1f%%" % (100 * h_drop.history["accuracy"]
   )[-1], 100 * h_drop.history["val_accuracy"][-1]))
plot_history(h_drop, "(dropout 0.3)")
```

The saved dropout run ends at 92.1% training accuracy and 89.4% validation accuracy. Its validation loss stays much lower than the late-epoch loss of the larger model without dropout. Dropout introduces a harder, noisy training calculation, which is one reason its training accuracy can be lower.

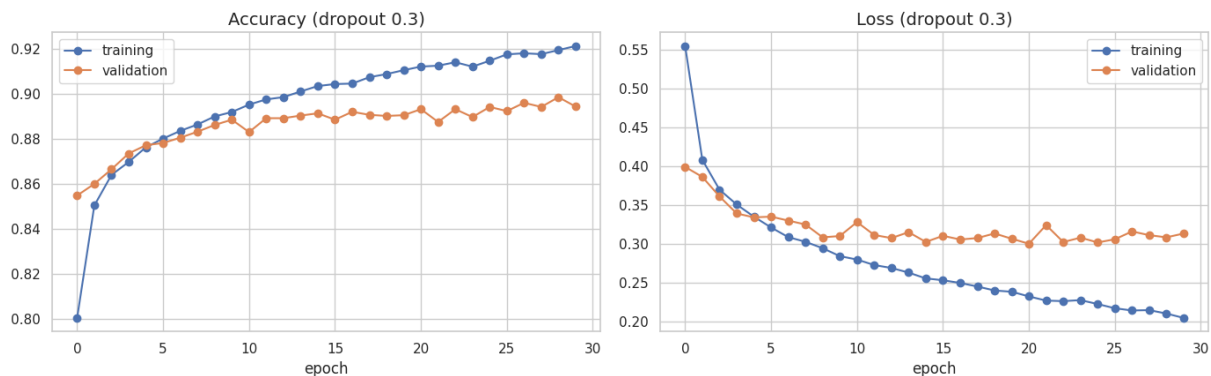


Figure 8: Saved training curves with dropout rate 0.3 after each hidden layer.

```
plt.figure(figsize=(9, 5))
plt.plot(h_big.history["val_loss"], "o-", label="no dropout")
plt.plot(h_drop.history["val_loss"], "o-", label="dropout 0.3")
plt.xlabel("epoch"); plt.ylabel("validation loss"); plt.legend(fontsize=12)
plt.title("Validation loss: lower is better", fontsize=14); plt.show()

for name, m in [("No dropout", big), ("Dropout 0.3", drop)]:
    print(f"{name:12s} test accuracy: {m.evaluate(X_test, y_test, verbose=0)[1]:.1%}")
```

The final comparison overlays the two validation-loss histories. The loop then evaluates both models on the test data, with dropout disabled during evaluation.

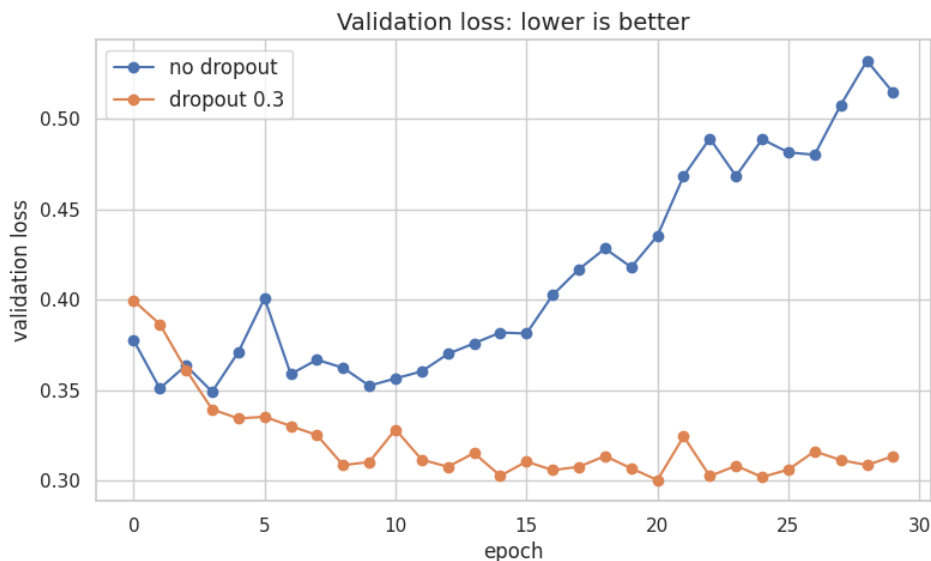


Figure 9: The saved validation-loss comparison. The late training epochs hurt the model without dropout.

Test accuracy	Slides/speaker notes	Saved notebook
Large model, no dropout	89.0%	88.5%
Large model, dropout 0.3	89.4%	89.1%

These runs illustrate a benefit under this setup. Dropout does not guarantee improvement on every dataset or architecture. Model size, data quality, learning rate, weight regularisation, and augmentation can all affect generalisation.

8.3 Exercise 2: 32 or 512 hidden neurons

```
# Exercise 2 (Dense 32): 784*32+32 + 32*10+10 = 25,450 weights, usually ~85-86% test accuracy.
# Dense 512: 407,050 weights, usually ~88-89%. Bigger helps a little, but costs more and
# overfits sooner.
small = keras.Sequential([keras.Input(shape=(28, 28)), layers.Flatten(),
                          layers.Dense(32, activation="relu"), layers.Dense(10, activation="softmax")])
small.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
small.fit(X_train, y_train, epochs=10, validation_split=0.1, verbose=0)
print("Dense 32 weights:", small.count_params(), " test acc: %.1f%%" % (100 * small.evaluate(X_test, y_test, verbose=0)[1]))
```

The solution constructs and trains the 32-neuron model. Its count is $(784+1)32+(32+1)10 = 25,450$. Changing the hidden width to 512 gives $(784+1)512+(512+1)10 = 407,050$.

The speaker notes report 86.5% test accuracy for the 32-neuron exercise and describe 88–89% as a typical expectation for width 512. There is no saved exercise output in the uploaded notebook, so these are attributed teaching results or expectations. Width alone cannot determine the eventual accuracy.

The supplied solution explicitly trains only the width-32 case. To run both choices consistently on validation data, create fresh models inside a loop, set the seed before each build, and record the validation results. Parameter count, runtime, and overfitting behaviour should be considered alongside accuracy.

8.4 Exercise 3: early stopping

```
# Exercise 3
es_model = big_model(0.0)
h_es = es_model.fit(X_train, y_train, epochs=30, batch_size=128, validation_split=0.1, verbose=0,
                    callbacks=[keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True)])
print("Stopped after", len(h_es.history["loss"]), "epochs; test acc %.1f%%" % (100 * es_model.evaluate(X_test, y_test, verbose=0)[1]))
```

A **callback** is code the training procedure calls at defined points, such as the end of each epoch. **EarlyStopping** monitors validation loss by default. With **patience=3**, it tolerates three consecutive epochs without sufficient improvement in the monitored quantity before stopping.

restore_best_weights=True restores the weights from the best monitored epoch after stopping. The printed history length is the number of epochs actually completed. It can be smaller than the requested maximum of 30.

The speaker notes report stopping after five epochs with 86.7% test accuracy in their exercise run. The uploaded notebook has no stored execution output for that exercise. No particular stopping epoch is guaranteed. A small patience can stop during an early fluctuation; increasing patience lets the model wait longer for improvement.

Choose patience and other settings using validation evidence. Evaluating many stopping strategies against the test set and selecting the best would reuse the test set for development.

9 Notebook 2: images, channels, and local structure

Material covered: Slides 17–18; Notebook 2, Parts 1–2.

9.1 Installing and importing the tools

```
!pip install -q ultralytics
```

The leading exclamation mark runs a shell command from the notebook. **pip install** installs a Python package, and **-q** reduces the amount of installation output. This is notebook syntax; a normal Python script would run installation separately.

The command does not pin a package version, so a later installation can behave differently. For a repeatable class, record a version you have tested. The saved transfer-learning logs identify Ultralytics 8.4.163. This identifies the environment of the supplied run.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import urllib.request
import cv2
import tensorflow as tf
import keras
from keras import layers
sns.set_theme(style="white")
keras.utils.set_random_seed(42)
print("GPU available:", len(tf.config.list_physical_devices("GPU")) > 0)
```

cv2 is the Python module name for OpenCV. It handles image loading, colour conversions, and filters in the practical examples. The TensorFlow and Keras imports support the small CNN. The separate YOLO commands later use Ultralytics and PyTorch.

9.2 What a colour image contains

A colour image commonly uses three channel values per pixel: red, green, and blue. A value such as [172, 146, 119] describes one pixel's channel intensities in RGB order. It is not three adjacent pixels.

An array shaped $(H, W, 3)$ has H rows, W columns, and three channel values at each position. A phone image with 4,000 by 3,000 pixels therefore contains $4000 \times 3000 \times 3 = 36$ million channel values. For uint8 data, those values occupy about 36 MB before overhead; compressed image files can be much smaller.

```
url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/bus.jpg"
urllib.request.urlretrieve(url, "bus.jpg")

img = cv2.imread("bus.jpg") # OpenCV loads colours as B, G, R
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # convert to the normal R, G, B order
print("Shape (height, width, channels):", img.shape)
print("Total numbers:", img.size)
print("Top-left pixel [R, G, B]:", img[0, 0])

plt.figure(figsize=(6, 8)); plt.imshow(img); plt.axis("off"); plt.show()
```

urlretrieve downloads the example to a local file. cv2.imread decodes its contents into an array. In this normal colour-loading mode, OpenCV returns the channels in BGR order. Matplotlib expects RGB, so cvtColor reorders them before display.

The supplied bus image has shape (1080, 810, 3) and 2,624,400 channel values. img.size counts all entries across all dimensions. img[0,0] selects the first row and first column and returns its three channel values: [172, 146, 119] after conversion.

If loading fails, imread can return None. Check the path and download before attempting further image operations. A colour-conversion error can be a symptom of a failed read.

```
# The three colour channels, shown separately
fig, axes = plt.subplots(1, 3, figsize=(15, 6))
for i, (ax, name, cmap) in enumerate(zip(axes, ["Red", "Green", "Blue"], ["Reds", "Greens", "Blues"])):
    ax.imshow(img[:, :, i], cmap=cmap); ax.set_title(name + " channel", fontsize=14); ax.axis("off")
plt.tight_layout(); plt.show()
```

`img[:, :, i]` takes every row and column from channel i . The loop draws each channel separately using a named colour map. These panels visualise channel intensity; the plotting colours are selected by Matplotlib.



Figure 10: Saved views of the red, green, and blue channels of the bus photograph.

A dense hidden layer of 128 units connected to the flattened full photograph would require

$$2,624,400 \times 128 + 128 = 335,923,328$$

parameters in that layer alone. Local filters share their parameters across many positions, which makes a different and much smaller connection scheme possible.

10 Convolution, worked out by hand

Material covered: Slide 19; Notebook 2, hand-built filters.

10.1 A filter produces one number from a local patch

A **filter** or **kernel** is a small array of weights. Place it over a matching image patch, multiply corresponding entries, and add the products. That total becomes one entry of an output **feature map**. Move to the next position and repeat.

The example filter is

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}.$$

It subtracts the left-column intensities from the right-column intensities and ignores the centre column. A dark-to-bright transition from left to right produces a positive response.

For the patch

$$P = \begin{bmatrix} 0 & 0 & 9 \\ 0 & 0 & 9 \\ 0 & 0 & 9 \end{bmatrix},$$

the result is $(-1)(0) + 0(0) + 1(9)$ on each row, giving $9 + 9 + 9 = 27$. For a patch filled entirely with nine, each row gives $-9 + 0 + 9 = 0$, so the output is zero. Reversing the bright and dark sides gives -27 .

This detector responds to an intensity change; it does not know whether that change belongs to a person, a window, or a bus. Learned networks combine many such local responses to build task-relevant representations.

10.2 The complete tiny-image calculation

```
vertical_edge = np.array([[ -1, 0, 1],
                          [ -1, 0, 1],
                          [ -1, 0, 1]], dtype=np.float32)
horizontal_edge = vertical_edge.T      # flip it: finds horizontal edges

# A tiny 6x6 test image: dark left half, bright right half
tiny = np.array([[0, 0, 0, 9, 9, 9]] * 6, dtype=np.float32)
out = cv2.filter2D(tiny, -1, vertical_edge, borderType=cv2.BORDER_CONSTANT)
print("Image:\n", tiny)
print("Feature map (big numbers = edge found):\n", out)
```

The array literal creates a six-by-six image with three dark columns and three bright columns. `dtype=np.float32` allows positive and negative floating-point responses. The transpose `.T` exchanges rows and columns of the filter to create the horizontal-edge detector. The operation exchanges the two spatial axes.

`filter2D` slides the supplied kernel over the image. The second argument `-1` requests the input's output depth, which is `float32` here. `BORDER_CONSTANT` supplies zero values outside the array so the output can keep the same spatial dimensions.

The independently checked feature map is

$$\begin{bmatrix} 0 & 0 & 18 & 18 & 0 & -18 \\ 0 & 0 & 27 & 27 & 0 & -27 \\ 0 & 0 & 27 & 27 & 0 & -27 \\ 0 & 0 & 27 & 27 & 0 & -27 \\ 0 & 0 & 27 & 27 & 0 & -27 \\ 0 & 0 & 18 & 18 & 0 & -18 \end{bmatrix}.$$

The central rows have responses of 27 at the transition. At the top and bottom, the filter overlaps only two nonzero image rows, giving 18. The far-right negative response comes from the artificial transition between the bright image and the zero padding outside it.

The operation here applies the kernel without reversing its entries. Its precise mathematical name is cross-correlation. Image libraries and neural-network discussions commonly use the word convolution for this learned sliding-filter operation. The multiply-and-add procedure above describes what the code actually performs.

10.3 Applying both filters to the photograph

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY).astype(np.float32)

v = np.abs(cv2.filter2D(gray, -1, vertical_edge))
h = np.abs(cv2.filter2D(gray, -1, horizontal_edge))

fig, axes = plt.subplots(1, 3, figsize=(16, 7))
axes[0].imshow(gray, cmap="gray"); axes[0].set_title("Greyscale", fontsize=14)
axes[1].imshow(v, cmap="gray"); axes[1].set_title("Vertical edges", fontsize=14)
axes[2].imshow(h, cmap="gray"); axes[2].set_title("Horizontal edges", fontsize=14)
for a in axes: a.axis("off")
plt.tight_layout(); plt.show()
```

The image is first converted to greyscale. That conversion combines the colour channels with luminance-related weights, producing one intensity value per position. Conversion to float32 preserves negative filter responses.

The vertical and horizontal filters are applied separately. `np.abs` takes the absolute value so both dark-to-bright and bright-to-dark transitions appear as positive strengths. This discards the direction of the brightness change while retaining its magnitude.



Figure 11: Saved greyscale, vertical-edge, and horizontal-edge views.

Upright window boundaries and poles respond strongly to the vertical-edge filter. Roof lines and other horizontal boundaries respond strongly to the transposed filter. The default border handling in this photo call differs from the explicit zero-padding option in the tiny example.

In a trained CNN, filter coefficients are learned through the same loss and gradient process used for dense weights. Hand-designed filters make the mechanism visible; the network learns coefficients that serve its prediction task.

10.4 Exercise 1: blur and sharpening

```
# Exercise 1
blur = cv2.filter2D(gray, -1, np.ones((5, 5), np.float32) / 25)
sharp = cv2.filter2D(gray, -1, np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]], np.float32))
fig, axes = plt.subplots(1, 2, figsize=(12, 7))
axes[0].imshow(blur, cmap="gray"); axes[0].set_title("Blur"); axes[1].imshow(np.clip(sharp, 0,
    255), cmap="gray"); axes[1].set_title("Sharpen")
for a in axes: a.axis("off")
plt.show()
```

The blur kernel contains 25 entries, each $1/25$. Its output is the mean of a five-by-five neighbourhood. A bright isolated pixel is spread across neighbouring outputs, making detail softer.

The sharpening kernel gives the centre a weight of five and subtracts the four immediate horizontal and vertical neighbours. A constant region with intensity a produces $5a - 4a = a$, while local changes receive stronger emphasis.

Sharpening can produce values below zero or above 255 in the floating-point result. `np.clip(sharp, 0, 255)` limits them for display. The operation can strengthen noise as well as meaningful boundaries. The supplied exercise has code and explanatory expectations but no saved output figure in this notebook.

11 Pooling and the complete CNN architecture

Material covered: Slides 20–22; Notebook 2, pooling and Part 3.

11.1 Max pooling with a numerical example

Max pooling summarises a local region by its largest activation. For a two-by-two pool with stride two, non-overlapping blocks are reduced to one value each:

$$\begin{bmatrix} 1 & 3 & 2 & 0 \\ 5 & 2 & 1 & 7 \\ 0 & 1 & 4 & 2 \\ 6 & 2 & 3 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 5 & 7 \\ 6 & 4 \end{bmatrix}.$$

The upper-left block contains 1, 3, 5, 2, whose maximum is five. Repeat for the other three blocks. The height and width are each halved, so the number of spatial values becomes one quarter.

```
fm = np.array([[1, 3, 2, 0],
               [5, 2, 1, 7],
               [0, 1, 4, 2],
               [6, 2, 3, 1]])
pooled = fm.reshape(2, 2, 2, 2).max(axis=(1, 3))
print("Feature map:\n", fm)
print("After 2x2 max pooling:\n", pooled)
```

`reshape(2,2,2,2)` groups the row and column structure into block indices and within-block positions. Taking the maximum over axes one and three reduces the within-block dimensions, leaving one maximum for each block. This particular reshape is chosen for the four-by-four example.

Pooling has no learned weights in this form. It reduces the size of later representations and may make a strong response less sensitive to a small movement within its region. It also discards

precise position information. Pooling does not guarantee that every small shift leaves the network's prediction unchanged.

11.2 Reloading Fashion-MNIST and adding a channel axis

```
import gzip, urllib.request

BASE = "https://raw.githubusercontent.com/zalando-research/fashion-mnist/master/data/fashion/"

def load_fashion(kind):
    """Download one half (train or t10k) of Fashion-MNIST straight from the official GitHub repo
    ."""
    raw_imgs = urllib.request.urlopen(BASE + f"{kind}-images-idx3-ubyte.gz").read()
    raw_labs = urllib.request.urlopen(BASE + f"{kind}-labels-idx1-ubyte.gz").read()
    imgs = np.frombuffer(gzip.decompress(raw_imgs), np.uint8, offset=16).reshape(-1, 28, 28)
    labs = np.frombuffer(gzip.decompress(raw_labs), np.uint8, offset=8)
    return imgs, labs

X_train_raw, y_train = load_fashion("train")
X_test_raw, y_test = load_fashion("t10k")

class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

print("Training images:", X_train_raw.shape) # (60000, 28, 28)
print("Test images:    ", X_test_raw.shape)  # (10000, 28, 28)
```

Notebook 2 repeats the loader because it should run independently. The decompression, byte offsets, image reshaping, and class codes have the same meanings as in Notebook 1. The standard training and test arrays are unchanged.

```
# CNNs expect a channel dimension: (28, 28) -> (28, 28, 1) (1 channel = greyscale)
X_train = (X_train_raw.astype("float32") / 255.0)[..., None]
X_test = (X_test_raw.astype("float32") / 255.0)[..., None]
print(X_train.shape)
```

The CNN expects height, width, and channels for each example. A greyscale image has one channel, so each image becomes (28, 28, 1). The full training shape is (60000, 28, 28, 1).

The ellipsis in `[..., None]` preserves all existing dimensions. `None` inserts a new length-one axis at the end. No new pixel information is invented; the array now states the channel dimension explicitly. This is the channels-last representation used by the example.

11.3 Building the CNN

```
keras.utils.set_random_seed(42)
cnn = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu", name="conv1"),
    layers.MaxPooling2D(2, name="pool1"),
    layers.Conv2D(64, 3, activation="relu", name="conv2"),
    layers.MaxPooling2D(2, name="pool2"),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax"),
])
cnn.summary()
```

`Conv2D(32, 3)` creates 32 filters with three-by-three spatial size. Each filter covers all input channels and produces one output feature map. `activation="relu"` applies ReLU after its weighted calculation. The layer name `conv1` makes it easy to retrieve later.

`MaxPooling2D(2)` reduces each map using two-by-two regions. The second convolution creates 64 maps from the 32-channel representation it receives, followed by another pooling step. Flattening then feeds the maps into a dense hidden layer, dropout, and the ten-class output.

The convolution defaults here use stride one and valid padding. Valid padding adds no border values, so a three-by-three filter shrinks each spatial dimension by two. For the usual undilated case,

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1,$$

where H is input height, P padding per side, K kernel size, and S stride. The floor symbol rounds down. Apply the same formula to width.

For the first layer, $H = 28$, $P = 0$, $K = 3$, $S = 1$, giving 26. Pooling gives 13. The second convolution gives 11; pooling with a two-by-two window and stride two gives five. An unmatched boundary position can be dropped under valid pooling.

Layer	Output per image	Parameters
Input	$28 \times 28 \times 1$	0
Conv1: 32 filters	$26 \times 26 \times 32$	320
Pool1	$13 \times 13 \times 32$	0
Conv2: 64 filters	$11 \times 11 \times 64$	18,496
Pool2	$5 \times 5 \times 64$	0
Flatten	1,600	0
Dense ReLU	128	204,928
Dropout	128	0
Dense softmax	10	1,290
Total		225,034

11.4 Counting convolution parameters

For a convolution with square kernel size K , C_{in} input channels, and C_{out} filters, the count including one bias per filter is

$$(K^2 C_{\text{in}} + 1) C_{\text{out}}.$$

The first layer has $(3 \times 3 \times 1 + 1)32 = 320$ parameters. The second has $(3 \times 3 \times 32 + 1)64 = 18,496$. The second-layer filter combines a small spatial neighbourhood from every one of its 32 input channels.

The same filter coefficients are reused at each spatial position. Moving the filter does not create another set of learned weights. This is parameter sharing. Its spatial output can be large while its learned parameter count remains modest.

After pooling, the flattened length is $5 \times 5 \times 64 = 1,600$. The hidden dense layer therefore has $(1600 + 1)128 = 204,928$ parameters, which accounts for most of this model. Dropout changes training behaviour but adds no learned coefficients.

This CNN has more total parameters than the small 101,770-parameter dense model. The advantage demonstrated here comes from its local structure and shared filters, not a lower total parameter count in this particular comparison.

11.5 Training and reading the result

```
cnn.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
h_cnn = cnn.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1)
```

The loss and optimiser have the same roles as before. Batch size 64 gives $\lceil 54000/64 \rceil = 844$ updates per epoch. The split still holds out 6,000 images for validation, and training continues for ten epochs.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
axes[0].plot(h_cnn.history["accuracy"], "o-", label="training"); axes[0].plot(h_cnn.history["val_accuracy"], "o-", label="validation")
axes[0].set_title("CNN accuracy", fontsize=14); axes[0].set_xlabel("epoch"); axes[0].legend(); axes[0].grid(True)
axes[1].plot(h_cnn.history["loss"], "o-", label="training"); axes[1].plot(h_cnn.history["val_loss"], "o-", label="validation")
axes[1].set_title("CNN loss", fontsize=14); axes[1].set_xlabel("epoch"); axes[1].legend(); axes[1].grid(True)
plt.tight_layout(); plt.show()

test_acc = cnn.evaluate(X_test, y_test, verbose=0)[1]
print(f"CNN test accuracy: {test_acc:.1%} (dense network in Notebook 1: about 87%)")
```

The plotting cell draws training and validation histories and then evaluates test accuracy. Index one selects accuracy from the returned loss-and-metric values.

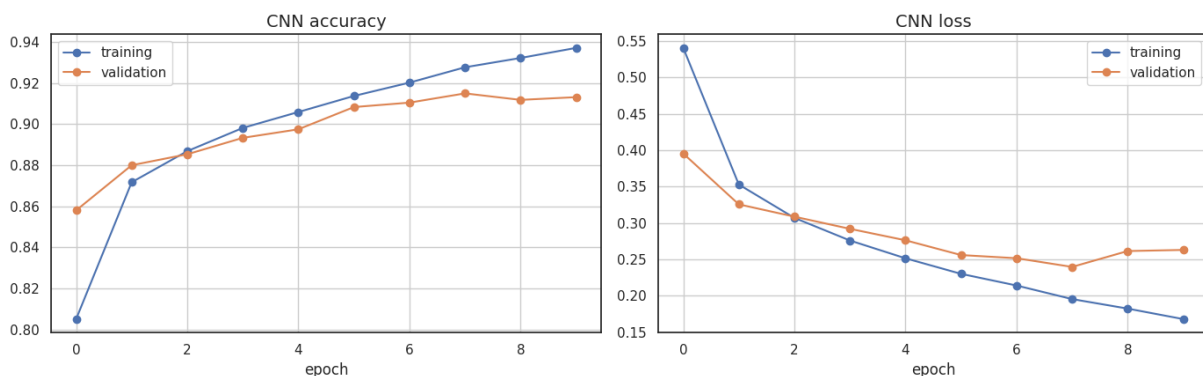


Figure 12: The saved CNN learning curves. The displayed epoch positions begin at zero.

The saved run reaches 93.7% training accuracy and 91.3% validation accuracy at the final epoch. Validation loss bottoms near epoch eight, then increases. Its printed test score is 90.9%. The slides describe a separate run with 91.0%.

The slides compare 87.1% dense accuracy with 91.0% CNN accuracy on 10,000 images. These rounded values correspond to about 1,290 and 900 errors. The relative reduction is

$$\frac{1290 - 900}{1290} \times 100 \approx 30.2\%.$$

The accuracy increase is 3.9 percentage points. “Thirty percent fewer errors” and “3.9 percentage points more accuracy” describe different calculations from the same reported comparison.

Both models see ten epochs, but the training configurations also differ in batch size, dropout, architecture, computation, and number of updates. The classroom comparison illustrates the full configurations. It does not isolate convolution as the only changed factor.

11.6 Checking the hard-coded comparison table

```
from sklearn.metrics import confusion_matrix
y_pred = cnn.predict(X_test, verbose=0).argmax(axis=1)
cm = confusion_matrix(y_test, y_pred)
per_class = cm.diagonal() / cm.sum(axis=1)
dense_nb1 = [91.5, 96.4, 81.7, 93.6, 72.5, 95.6, 53.8, 96.9, 95.6, 93.7] # from Notebook 1 (
    our run)
print(f"{'class':11s} dense   CNN")
for name, d, c in zip(class_names, dense_nb1, per_class):
    print(f"{'name':11s} {d:5.1f}% {100*c:5.1f}%")
```

The CNN recalls are calculated from its current confusion matrix. The dense values in `dense_nb1` are a fixed list copied from the earlier run used in the slides. They do not automatically update when you rerun Notebook 1.

Here are the two supplied records in matching class order:

Class	Dense slides	CNN slides	Dense saved	CNN saved
T-shirt	91.5	89.5	90.2	88.9
Trouser	96.4	98.0	95.8	97.6
Pullover	81.7	86.3	80.3	91.5
Dress	93.6	93.1	94.2	92.9
Coat	72.5	86.5	75.4	87.3
Sandal	95.6	98.7	96.1	98.9
Shirt	53.8	68.6	54.1	62.1
Sneaker	96.9	97.9	97.1	97.0
Bag	95.6	97.9	95.1	98.4
Ankle boot	93.7	93.8	94.1	94.8

All entries are percentages and represent recall for the named class. Improvements are uneven: the CNN helps some difficult classes while other recalls decline. Overall accuracy can increase even when a particular class becomes less accurate.

For a comparison of your own runs, save the dense model’s calculated recall array and use those actual values. Keep the split, labels, and class order consistent.

12 Looking inside a CNN and extending it

Material covered: Notebook 2, feature maps, learned filters, and Exercise 2.

12.1 Building a model that stops at an intermediate layer

```
viewer = keras.Model(inputs=cnn.inputs, outputs=cnn.get_layer("conv1").output)
i = 0 # first test image (try others!)
maps = viewer.predict(X_test[i:i+1], verbose=0)[0] # shape (26, 26, 32)

plt.figure(figsize=(3, 3)); plt.imshow(X_test[i, :, :, 0], cmap="gray_r"); plt.title(class_names[
    y_test[i]]); plt.axis("off"); plt.show()
plt.figure(figsize=(14, 7))
for k in range(32):
    plt.subplot(4, 8, k + 1); plt.imshow(maps[:, :, k], cmap="viridis"); plt.axis("off")
plt.suptitle("The 32 feature maps from conv1 (bright = filter fired)", fontsize=14)
plt.tight_layout(); plt.show()
```

`cnn.get_layer("conv1")` retrieves the named first convolution. The new Keras Model uses the original input and that layer's output, allowing us to inspect activations inside the trained network. It reuses the existing learned layer; it does not train another CNN.

The slice `X_test[i:i+1]` keeps a batch dimension of one. Using `X_test[i]` alone would remove that dimension. After prediction, `[0]` removes the single batch entry, leaving maps with shape `(26, 26, 32)`.

The first figure shows the input ankle boot. The loop then draws all 32 channel maps. Bright areas have stronger activation under the panel's colour scale. Dark areas have smaller or zero responses after ReLU.

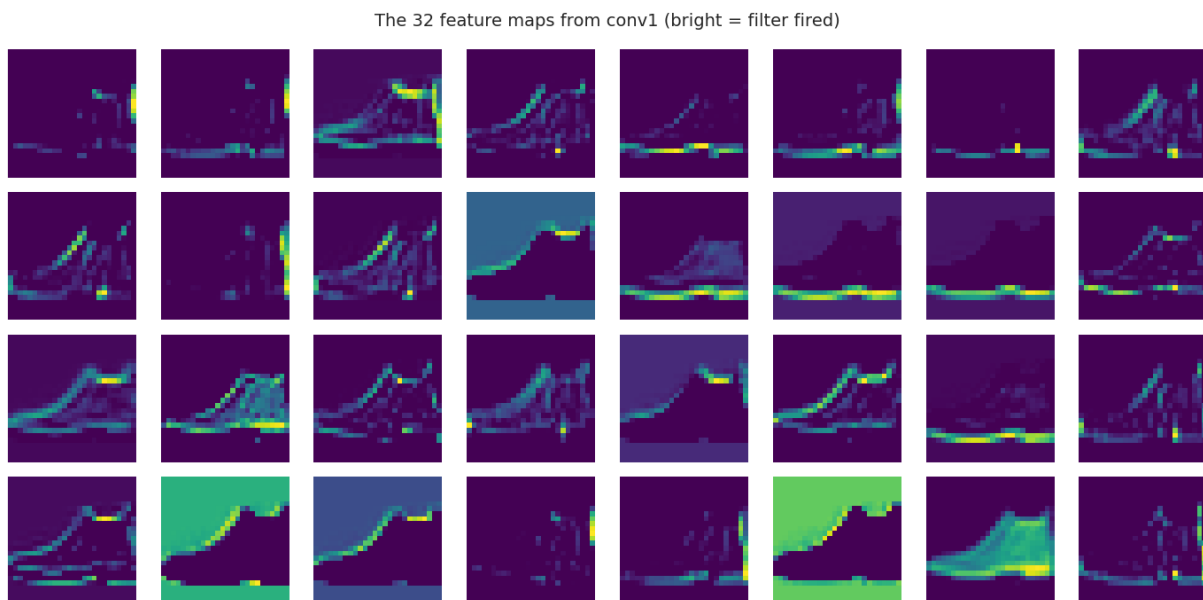


Figure 13: The 32 saved first-layer feature maps for one ankle-boot image.

Some maps respond to the outline or sole, others to broader regions or the background. The plots demonstrate different numerical responses, without proving that each filter represents a specific named concept. Default per-panel colour scaling can also exaggerate weak responses, so compare actual ranges if making a quantitative claim.

12.2 The filter weights themselves

```
# The learned 3x3 filters themselves
w = cnn.get_layer("conv1").get_weights()[0] # shape (3, 3, 1, 32)
plt.figure(figsize=(14, 4))
for k in range(32):
    plt.subplot(2, 16, k + 1); plt.imshow(w[:, :, 0, k], cmap="coolwarm"); plt.axis("off")
plt.suptitle("32 learned 3x3 filters (red = positive weight, blue = negative)", fontsize=14)
plt.show()
```

`get_weights()` returns the learned kernel array and bias array for this layer. Index zero chooses the kernels. Their shape is (3,3,1,32): height, width, input channels, output filters.

The slice `w[:, :, 0, k]` selects the three-by-three spatial weights for input channel zero and filter k . Here there is only one input channel. The feature map is the result of applying these coefficients to an image and then applying the layer's bias and activation.

32 learned 3x3 filters (red = positive weight, blue = negative)

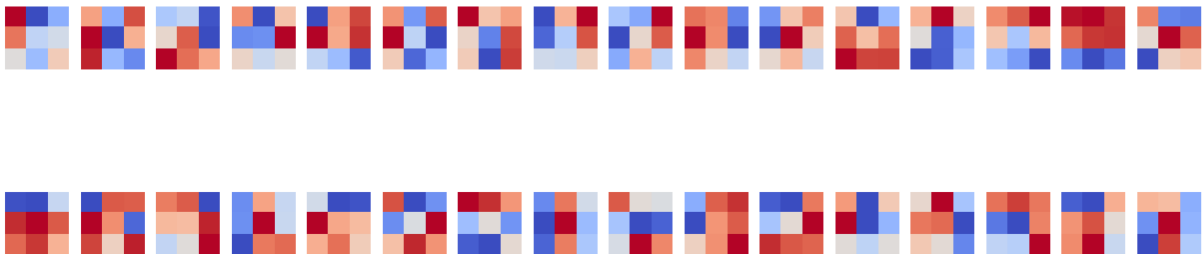


Figure 14: The saved learned filter coefficients. Each small square contains nine coefficients.

The plotting title describes red as positive and blue as negative. Because `imshow` scales each panel to its own minimum and maximum by default, the colours reliably indicate relatively larger and smaller values. To make colour zero-centred and directly comparable, use one symmetric range for all panels, such as `vmin=-limit, vmax=limit`, with `limit=np.abs(w).max()`.

12.3 Exercise 2: add a third convolution

```
# Exercise 2: in our run the third conv layer did NOT help (90.4% vs 91.0%) - deeper is not
# automatically better.
# Weights change: conv3 adds 73,856, but the flatten output shrinks (3x3x128 = 1,152 instead of
# 1,600), so the dense layer gets smaller.
keras.utils.set_random_seed(42)
cnn3 = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu"), layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, activation="relu"), layers.MaxPooling2D(2),
    layers.Conv2D(128, 3, activation="relu"),
    layers.Flatten(), layers.Dense(128, activation="relu"), layers.Dropout(0.3),
    layers.Dense(10, activation="softmax")])
cnn3.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
cnn3.fit(X_train, y_train, epochs=10, batch_size=64, validation_split=0.1, verbose=0)
print("Weights:", cnn3.count_params(), " test acc: %.1f%%" % (100 * cnn3.evaluate(X_test, y_test
, verbose=0)[1]))
```

The new layer receives $5 \times 5 \times 64$ values after pool2. A valid three-by-three convolution with 128 filters produces $3 \times 3 \times 128$. No pooling is added afterward, so the flattened representation has $3 \times 3 \times 128 = 1,152$ entries.

The new convolution adds $(3 \times 3 \times 64 + 1)128 = 73,856$ parameters. The dense hidden layer becomes smaller, with $(1152 + 1)128 = 147,584$. Compared with its earlier 204,928 parameters, this removes 57,344. The net increase is therefore only $73,856 - 57,344 = 16,512$, giving a new total of 241,546.

The speaker notes report 90.4% test accuracy for their third-convolution experiment, against their original 91.0%. No output for this exercise is stored in the uploaded notebook. The parameter count follows exactly from the code, while accuracy requires a training run.

Additional depth changes representation and optimisation. The smaller spatial map also discards or combines more detail. Evaluate whether the new architecture helps using validation evidence, and consider its training schedule and regularisation.

13 Transfer learning: starting with learned weights

13.1 What is transferred?

A freshly created network starts with initial numerical weights. Training changes those numbers using examples and a loss. A pretrained network already contains weights learned during an earlier training task. Transfer learning uses those weights as a starting point for a related task.

Early image layers often respond to edges, colour changes and small textures. Later layers combine these into more complex patterns. These representations can be useful when a new dataset has relatively few examples. The usefulness depends on how closely the earlier images and task relate to the new ones.

There are two common training arrangements. Feature extraction freezes the pretrained feature layers and trains a new output head. Fine-tuning allows some or all pretrained layers to change as well. The notebook uses fine-tuning: its saved training log shows the model parameters remain trainable. Loading pretrained weights does not automatically freeze them.

13.2 The Imagenette experiment

Imagenette contains ten selected ImageNet categories: tench, English springer spaniel, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball and parachute. The training folder contains 9,469 images and the validation folder contains 3,925 images. A folder's name supplies the class label for the images inside it.

The experiment trains one classifier from pretrained weights and another from a model architecture with newly initialised weights. Both receive the same short training budget. This examines the benefit of the starting weights during this run. The pretrained model has already benefited from earlier training, so the two models have different total training histories.

Imagenette is derived from ImageNet, and the pretrained classifier was trained for ImageNet classification. This is a closely related demonstration. It should not be presented as evidence of performance on a completely unfamiliar domain or as an independently audited evaluation free of overlap with pretraining data.

13.3 Training the pretrained classifier

```
# We run YOLO from the command line ("!yolo ...") so it trains in its own process,  
# separate from TensorFlow. Same thing as YOLO("yolo11n-cls.pt").train(...) in Python.  
!yolo classify train model=yolo11n-cls.pt data=imagenette160 epochs=2 imgsz=128 batch=64  
    fraction=0.5 seed=42 plots=False name=pretrained exist_ok=True
```

The exclamation mark sends a command to the notebook's shell. `yolo` is the command-line program installed by Ultralytics. `classify` chooses whole-image classification and `train` starts training. Running the command in another process also separates this training job from the TensorFlow process used earlier.

`model=yolo11n-cls.pt` loads pretrained classification weights. The `n` identifies the nano model size; `cls` identifies the classification task. A `.pt` file stores a PyTorch checkpoint. The program can download a recognised model name when the file is absent.

`data=imagenette160` selects the dataset. `epochs=2` makes two training passes. `imgsz=128` sets the classifier's requested input size. Images undergo the classification preprocessing pipeline, so this is not a promise that every original image is simply stretched without cropping or augmentation. `batch=64` processes up to 64 training examples per update.

`fraction=0.5` uses half the training examples: the saved log reports 4,734. It does not halve the validation set. `seed=42` controls seeded randomness, although exact reproducibility also depends on software, hardware and deterministic operations. `plots=False` suppresses training plots. `name=pretrained` names the run directory. `exist_ok=True` permits reuse of that directory, so separate names are helpful when preserving multiple experiments.

The classification head is adjusted to ten outputs. The saved model summary after this adjustment contains 1,543,914 parameters. The 2.6-million figure used elsewhere in the slides describes the nano detection model, which has a different task head and architecture.

13.4 Training from initial weights

```
# Same architecture (.yaml = just the blueprint), random starting weights  
!yolo classify train model=yolo11n-cls.yaml pretrained=False data=imagenette160 epochs=2 imgsz  
    =128 batch=64 fraction=0.5 seed=42 plots=False name=scratch exist_ok=True
```

The `.yaml` file describes the network structure. `pretrained=False` requests training without pretrained weights. The remaining settings match the preceding experiment, with `name=scratch` keeping its results in a separate run folder.

A short two-epoch run gives the initialised model little time to learn useful image features. Its lower accuracy in this example is evidence about this training budget and dataset. It does not establish an upper limit on what the architecture could learn with more data, tuning and training.

13.5 Reading and comparing the saved results

```
import pandas as pd
res = {}
for name in ["pretrained", "scratch"]:
    df = pd.read_csv(f"runs/classify/{name}/results.csv")
    df.columns = df.columns.str.strip()
    res[name] = df["metrics/accuracy_top1"].iloc[-1]
    print(f"{name:10s} accuracy after each epoch:", df["metrics/accuracy_top1"].round(3).tolist())

plt.figure(figsize=(8, 4))
bars = plt.barh(["From scratch", "Pre-trained"], [100 * res["scratch"], 100 * res["pretrained"]], color=["#F28C28", "#245BBF"])
plt.bar_label(bars, fmt="%1f%%", fontsize=13)
plt.xlim(0, 105); plt.xlabel("validation accuracy (%)")
plt.title("Same model, same data, 2 epochs", fontsize=14); plt.show()
```

`pd.read_csv` loads each training log into a table. Stripping column names removes surrounding spaces that could otherwise prevent a column lookup. `metrics/accuracy_top1` is the fraction of validation images for which the highest-scoring class matches the label. `iloc[-1]` selects the final recorded epoch.

In the uploaded notebook, pretrained validation accuracy rises from 87.6% to 92.8%, while the initialised model rises from 17.1% to 28.4%. The slides describe an earlier run ending at approximately 92.2% and 30.0%. These are validation results. The notebook does not provide a separate final test set for this experiment.

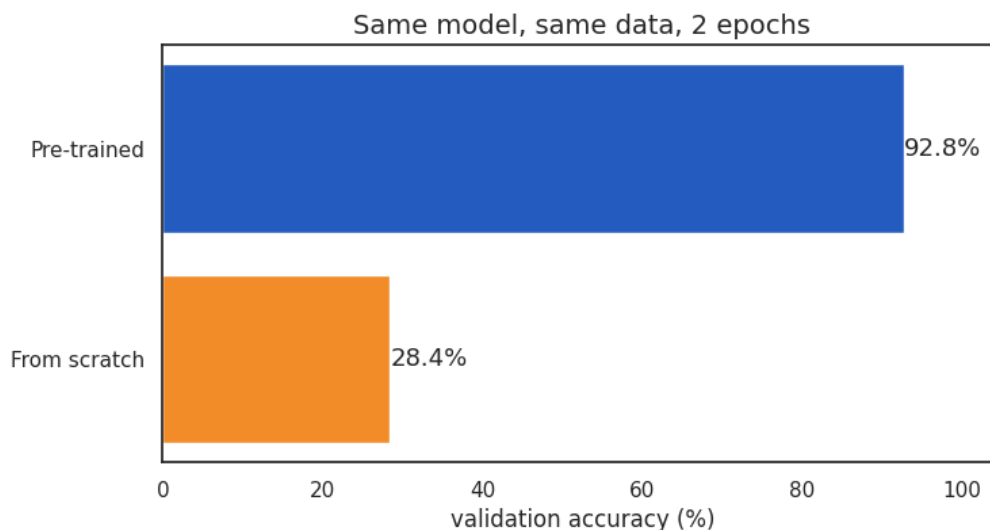


Figure 15: The comparison saved in Notebook 2: final-epoch validation accuracy after two epochs.

With ten equally likely classes, uniform random guessing has expected accuracy $1/10 = 10\%$. Actual class frequencies and prediction behaviour affect the accuracy of other simple baselines. The initialised model's result above 10% suggests it has learned useful information, even though this short run leaves substantial room for improvement.

13.6 Choosing images for a visual check

```
import glob, json, random, shutil, os
from pathlib import Path

# Find where Ultralytics saved the dataset
settings = json.load(open(Path.home() / ".config" / "Ultralytics" / "settings.json"))
val_dir = Path(settings["datasets_dir"]) / "imagenette160" / "val"

random.seed(1)
files = random.sample(sorted(glob.glob(str(val_dir / "*" / "*.JPEG"))), 8)
shutil.rmtree("samples", ignore_errors=True); os.makedirs("samples")
for f in files:
    shutil.copy(f, "samples/")

# The dataset's folders use ImageNet codes; the model uses these names
code2name = {"n01440764": "tench", "n02102040": "English_springer", "n02979186": "cassette_player",
             "n03000684": "chain_saw", "n03028079": "church", "n03394916": "French_horn",
             "n03417042": "garbage_truck", "n03425413": "gas_pump", "n03445777": "golf_ball", "n03888257": "parachute"}
```

`Path.home()` locates the user's home folder. The JSON settings file tells the script where Ultralytics placed its datasets. `json.load` converts its contents into a Python dictionary. `glob` finds validation image paths, and sorting creates a stable order before `random.sample` chooses eight distinct images. The seed makes that selection repeatable when the available paths are unchanged.

The cell removes and recreates a local folder called `samples`. Keep that folder for generated examples: rerunning the cell deletes its previous contents. `shutil.copy` copies the selected images into it. The `code2name` dictionary translates ImageNet folder identifiers into readable labels.

If this cell cannot find the settings file or validation images, inspect the actual dataset location from the training log. File locations can vary across installations. A successful training run must occur before the expected dataset and model paths can be used.

```
!yolo classify predict model=runs/classify/pretrained/weights/best.pt source=samples imgsz=128
    save=False save_txt=True name=pred exist_ok=True verbose=False
```

`predict` performs inference using the trained model. `best.pt` is the checkpoint selected as best during training. The earlier bar chart used the final CSV row, which is a distinct selection rule. `save_txt=True` writes predicted labels and scores, while `save=False` avoids saving annotated image copies. The output directory follows the task and run name, here `runs/classify/pred` under the notebook's settings.

```
plt.figure(figsize=(16, 7))
for k, f in enumerate(sorted(glob.glob("samples/*.JPEG"))):
    stem = Path(f).stem
    top = open(f"runs/classify/pred/labels/{stem}.txt").readline().split() # "probability
    class"
    prob, cls = float(top[0]), top[1]
    true = Path([g for g in files if Path(g).stem == stem][0]).parent.name
    plt.subplot(2, 4, k + 1)
    plt.imshow(cv2.cvtColor(cv2.imread(f), cv2.COLOR_BGR2RGB)); plt.axis("off")
    ok = cls == code2name[true]
    plt.title(f"{cls.replace('_', ' ')} ({prob:.0%})" + (" if ok else f"\nreally: {code2name[
    true].replace('_', ' ')}"),
            fontsize=12, color="green" if ok else "red")
plt.suptitle("Fine-tuned model on validation photos (green = right)", fontsize=14)
plt.tight_layout(); plt.show()
```

The plotting cell opens the saved prediction text, reads the leading prediction, and matches each sample's filename to its original validation folder. It displays the predicted class and uses colour to indicate whether the prediction agrees with the folder label. The comparison depends on the expected text format and class-name mapping; inspect those files if a different library version changes the format.

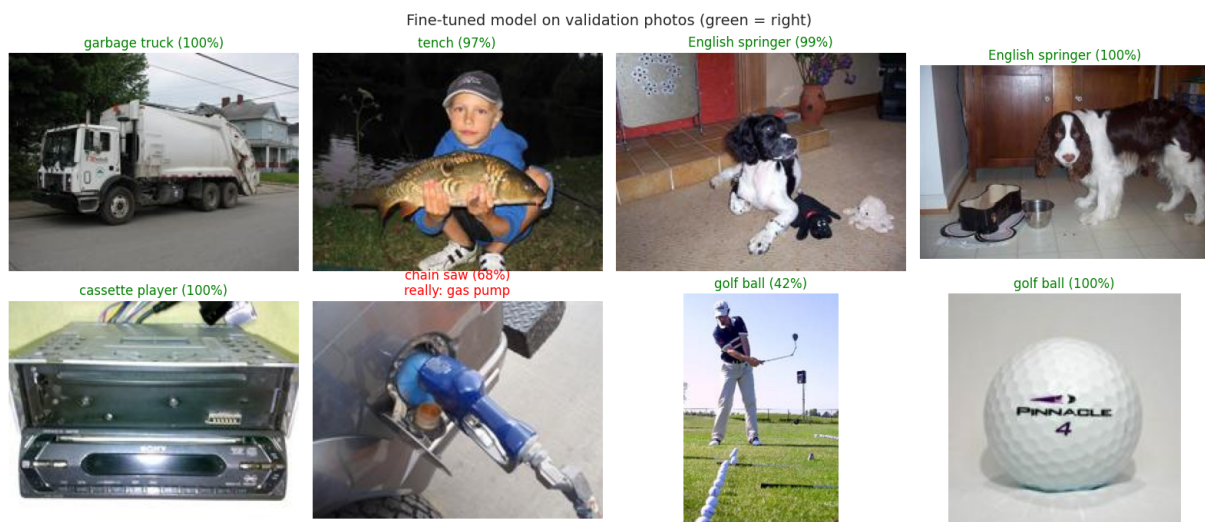


Figure 16: The eight sample predictions saved in the notebook. This small display helps inspect behaviour; it is not a substitute for evaluation on all validation examples.

Seven of the eight displayed examples are correct. In the stored figure, a gas-pump image is labelled as a chain saw. The speaker notes describe a different mistaken label from an earlier run. The image alone does not reveal exactly which features caused the error.

13.7 Optional longer run

```
# Exercise 3 (GPU recommended: ~3-4 minutes on T4). Remove the # to run.
# !yolo classify train model=yolo11n-cls.pt data=imagenette160 epochs=5 imgsz=160 batch=64
  fraction=1.0 seed=42 plots=False name=pre5 exist_ok=True
```

This line is commented out, so it does nothing until the comment marker is removed. It requests five epochs, input size 160 and the full training fraction. These changes increase the work and may

affect accuracy. There is no saved output for this optional run. The suggested accuracy in the teaching material is an expectation to investigate by running the experiment.

14 Computer vision: understanding the task

The third notebook uses pretrained models for inference. It loads learned weights and applies them to images; it does not train a new model.

Task	Output and meaning
Classification	A score for each class describing the whole image.
Object detection	A class, confidence score and rectangular box for each retained detection.
Instance segmentation	A class and pixel mask for each predicted object instance.
Pose estimation	Predicted body keypoint locations for each detected person.

Two people can have two separate masks despite sharing the same class. This is the meaning of *instance*. Semantic segmentation instead assigns category labels across pixels without necessarily separating individual objects of the same category.

14.1 Preparing the notebook

```
!pip install -q ultralytics
```

`pip install` installs the Ultralytics package and required dependencies. The `-q` option reduces installation output. This cell needs a working package source connection. A runtime restart can sometimes be necessary after changing packages in an already-running notebook.

```
import numpy as np
import matplotlib.pyplot as plt
import urllib.request, collections, time
import cv2
from ultralytics import YOLO

base = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/"
for f in ["bus.jpg", "zidane.jpg"]:
    urllib.request.urlretrieve(base + f, f)

def show(img_rgb, title="", size=(8, 8)):
    plt.figure(figsize=size); plt.imshow(img_rgb); plt.title(title, fontsize=14); plt.axis("off")
    plt.show()
print("Ready!")
```

NumPy handles arrays, Matplotlib displays images, and OpenCV supplies image operations. `urllib.request` downloads the two demonstration photographs. `collections` supplies the counting tool used later, and `time` supplies the timer. YOLO is the interface used to load task-specific models.

The loop joins a base URL with each filename and saves each file locally. Subsequent relative paths refer to this working directory. The helper `show` creates a figure, displays an RGB image, sets a title, hides the axes and displays the result. Its default figure size is a display setting, not an instruction to resize the image array.

15 OpenCV: pixels, transformations and drawing

15.1 Loading an image and ordering colour channels

```
bgr = cv2.imread("bus.jpg")
fig, axes = plt.subplots(1, 2, figsize=(12, 7))
axes[0].imshow(bgr); axes[0].set_title("Without converting (BGR) - looks blue!", fontsize=13)
rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
axes[1].imshow(rgb); axes[1].set_title("Converted to RGB", fontsize=13)
for a in axes: a.axis("off")
plt.show()
print("Size (height, width, channels):", rgb.shape)
```

`cv2.imread` returns an array in blue, green, red channel order. Matplotlib interprets a three-channel image as red, green, blue. Passing the first array directly therefore swaps the interpretation of red and blue. `cv2.cvtColor` rearranges the channels correctly.

The bus image has shape (1080, 810, 3): 1,080 rows, 810 columns and three colour values per pixel. Indexing uses `rgb[y, x]`, with row first. Drawing coordinates usually use (x, y) , with horizontal position first. The top-left pixel is near (0,0); moving right increases x and moving down increases y .

If `imread` cannot read a file, it returns `None`. Check the path and successful download before passing the result into a colour conversion.



Figure 17: The same stored colour values displayed with incorrect and correct channel interpretation.

15.2 Resize, grayscale, blur, edges and crop

```

small = cv2.resize(rgb, (270, 360))                # (width, height)
gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
blur = cv2.GaussianBlur(rgb, (21, 21), 0)
edges = cv2.Canny(gray, 100, 200)                  # the classic edge detector
crop = rgb[200:700, 0:400]                          # rows 200-700, columns 0-400 (just
    NumPy slicing!)

fig, axes = plt.subplots(1, 5, figsize=(20, 6))
for a, im, t in zip(axes, [small, gray, blur, edges, crop], ["Resized", "Grey", "Blurred", "
    Canny edges", "Cropped"]):
    a.imshow(im, cmap="gray" if im.ndim == 2 else None); a.set_title(t, fontsize=14); a.axis("
    off")
plt.tight_layout(); plt.show()

```

`resize(rgb, (270, 360))` requests width 270 and height 360. The result's shape is $(360, 270, 3)$. This preserves the original width-to-height ratio. Resizing estimates new pixel values through interpolation.

Grayscale combines the colour channels into one intensity channel. It reduces the array to two dimensions. A Gaussian blur computes weighted local averages, with greater weight near each pixel. The $(21, 21)$ neighbourhood controls its spatial extent, and the zero sigma argument asks OpenCV to derive the spread from the kernel size.

Canny edge detection uses intensity gradients, thinning and connected weak/strong-edge decisions. The values 100 and 200 are its lower and upper thresholds. Its bright output marks selected image boundaries. Edges alone do not assign object names.

`rgb[200:700, 0:400]` extracts rows 200 through 699 and columns 0 through 399, giving a $500 \times 400 \times 3$ crop. Python excludes the stop index. The cell comment's informal "200–700" should be read with that indexing rule.

`zip` pairs each plotting axis with one image and title. `im.ndim == 2` identifies a two-dimensional array and selects a grayscale colour map for its display. `tight_layout` adjusts subplot spacing.



Figure 18: The five image operations recorded in the notebook.

15.3 Drawing annotations

```
# Drawing: rectangles, circles and text (this is how detection boxes are drawn)
canvas = rgb.copy()
cv2.rectangle(canvas, (50, 230), (800, 750), (255, 140, 0), 6)           # (x1, y1), (x2, y2),
    colour, thickness
cv2.circle(canvas, (330, 850), 60, (36, 91, 191), 6)
cv2.putText(canvas, "BUS?", (60, 215), cv2.FONT_HERSHEY_SIMPLEX, 2.5, (255, 140, 0), 6)
show(canvas, "Drawn by hand with OpenCV")
```

`rgb.copy()` creates a separate array because the drawing functions modify their input. A rectangle uses two opposite corners, a colour tuple and line thickness. A circle uses a centre, radius, colour and thickness. `putText` takes the string, starting position, font, scale, colour and thickness.

Here the canvas is already RGB, so the supplied colour tuple becomes RGB when displayed. OpenCV drawing writes the provided channel values; it does not inspect the array and convert its colour order. The hand-drawn rectangle is an annotation chosen by the programmer. There is no model prediction in this cell.

15.4 Exercise 1: rotation and reflection

```
# Exercise 1
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].imshow(cv2.rotate(rgb, cv2.ROTATE_90_CLOCKWISE)); axes[0].set_title("Rotated")
axes[1].imshow(cv2.flip(rgb, 1)); axes[1].set_title("Flipped left-right")
for a in axes: a.axis("off")
plt.show()
```

A clockwise quarter-turn swaps the image's height and width. The rotated array therefore has shape (810,1080,3). `flip(rgb, 1)` reflects left and right while keeping the original dimensions. Flip code zero reflects top and bottom; minus one reflects both directions.

These operations can form part of data augmentation, where modified training examples help a model handle plausible variation. The transformation should preserve the intended label. For detection, boxes must move with the image; for segmentation, masks must move too; and for pose, keypoints and left/right conventions need corresponding updates.

16 Object detection with YOLO

16.1 Loading a detector and making predictions

```
model = YOLO("yolo11n.pt")          # downloads ~5 MB the first time
results = model("bus.jpg")           # run detection
show(results[0].plot()[:, :, ::-1], "YOLO11 detection") # .plot() draws the boxes (BGR -> RGB)
```

YOLO expands to “You Only Look Once”. The name describes a family of networks that predict object locations and classes through a unified detection pipeline. A call includes image preparation, network computation and postprocessing that selects the retained detections. It can miss visible objects or produce mistaken detections.

`yolo11n.pt` loads the nano detection model trained on the 80 COCO object categories. The notebook describes approximately 2.6 million parameters and a roughly 5 MB download. Those figures refer to this detection checkpoint.

`model("bus.jpg")` returns a collection of result objects. `results[0]` selects the result for the first image. `plot()` draws predicted annotations and returns a BGR image. The slice `[:, :, ::-1]` keeps rows and columns and reverses the channel axis to RGB for display.



Figure 19: Left: the programmer’s drawing. Right: the detections saved from the pretrained network.

16.2 Reading boxes, classes and scores

```
r = results[0]
print("Found", len(r.bboxes), "objects:")
for box, cls, conf in zip(r.bboxes.xyxy, r.bboxes.cls, r.bboxes.conf):
    x1, y1, x2, y2 = box.int().tolist()
    print(f" {r.names[int(cls)]:8s} confidence {conf:.2f} box ({x1},{y1}) -> ({x2},{y2})")
```

`r.bboxes.xyxy` contains corner coordinates (x_1, y_1, x_2, y_2) for each box. `r.bboxes.cls` contains numerical class identifiers. `r.bboxes.conf` contains their confidence scores. `zip` iterates over the corresponding entries together. `r.names` maps class identifiers to names.

`box.int()` converts the floating-point coordinates into integers for printing, discarding fractional parts. `tolist()` turns a tensor into ordinary Python values. The formatting `:.2f` displays two decimal places; `:8s` reserves at least eight character positions for a string.

The stored run retains five detections: one bus and four people. The bus confidence is about 0.94, with a printed box from (3, 229) to (796, 728). Its approximate width is $796 - 3 = 793$ pixels and height is $728 - 229 = 499$ pixels. A rectangular box contains background pixels as well as parts of the object.

Confidence is a model score used to rank or filter predictions. A score of 0.94 does not by itself establish that precisely 94% of similarly scored predictions will be correct; that interpretation requires calibration evidence.

16.3 Timing the complete call

```
# How fast is it?
t = time.time()
for _ in range(5):
    model("bus.jpg", verbose=False)
ms = (time.time() - t) / 5 * 1000
print(f"{ms:.0f} ms per image -> about {1000/ms:.0f} images per second")
```

The loop runs five calls, divides the elapsed seconds by five and multiplies by 1,000 to report milliseconds per image. Dividing 1,000 by that duration estimates sequential images per second. For example, 20 milliseconds corresponds to approximately 50 images per second.

This measures the surrounding prediction call, which may include loading and preprocessing the file, inference and postprocessing. The preceding call has already warmed the model. It is not a pure measurement of one neural network operation. Device, image size, software, power settings and timing methodology affect the result. The notebook's saved GPU result is about 17 milliseconds per image; the speaker notes describe a slower CPU example. Neither should be promised for another computer.

For a more stable benchmark, use more repeated calls, a high-resolution timer such as `time.perf_counter`, and a consistent device and input pipeline. GPU-only timing additionally requires handling asynchronous computation correctly.

16.4 Counting and confidence thresholds

```
# Counting objects, and filtering by confidence
r = model("zidane.jpg", conf=0.25, verbose=False)[0]
print(collections.Counter(r.names[int(c)] for c in r.bboxes.cls))
show(r.plot()[ :, :, ::-1], "zidane.jpg", size=(12, 7))
```

`collections.Counter` counts how often each predicted class name appears. The uploaded result for `zidane.jpg` contains two people and one tie. Their scores are approximately 0.84, 0.78 and 0.45. Raising `conf` from 0.25 to 0.6 removes the tie in this stored example.



Figure 20: The saved detections at the lower confidence threshold.

A higher threshold filters out more low-scoring predictions. This often reduces false positives and also removes some correct detections. Precision measures the proportion of retained detections that are correct; recall measures the proportion of relevant objects found under the chosen matching rules. Counting retained boxes is a count of predictions, which can differ from the true number of objects.

16.5 Exercise 2: keep only people

```
# Exercise 2
r = model("bus.jpg", classes=[0], verbose=False)[0]
print("People:", len(r.bboxes))
print(model.names)
```

`classes=[0]` keeps class zero for this detector, which its name mapping identifies as person. `len(r.bboxes)` counts the retained people boxes. The earlier saved bus-image prediction contained four people, so that provides a reference for this example. The exercise cell itself has no stored result. Always check the loaded model's `names`: a numerical class identifier has meaning within that model's label mapping.

17 Classification, masks and body keypoints

17.1 Whole-image classification

```
# Classification: one label for the whole picture (1,000 ImageNet classes)
cls_model = YOLO("yolo11n-cls.pt")
r = cls_model("bus.jpg", verbose=False)[0]
for i in r.probs.top5:
    print(f"{r.names[i]:15s} {r.probs.data[i]:.3f}")
```

The classification checkpoint ends in `-cls.pt`. Its original head contains 1,000 ImageNet categories. This is the original pretrained classifier, separate from the ten-class checkpoint fine-tuned in Notebook 2.

`r.probs` stores class probabilities. `top5` returns the indices of the five largest values. Each index selects a readable name from `r.names` and a score from `r.probs.data`.

The saved bus-photo scores are approximately 0.571 for minibus, 0.337 for police van, 0.042 for trolleybus, 0.014 for recreational vehicle and 0.006 for streetcar. The top five total about 0.970, with the remaining probability distributed across other classes. These are predictions about the whole image; they contain no separate box for each person.

17.2 Instance segmentation

```
# Segmentation: the exact pixels of each object
seg_model = YOLO("yolo11n-seg.pt")
r = seg_model("bus.jpg", verbose=False)[0]
print("Objects:", collections.Counter(r.names[int(c)] for c in r.bboxes.cls))
print("Masks shape:", tuple(r.masks.data.shape), "= (objects, height, width)")
show(r.plot()[:, :, :-1], "YOLO11 segmentation")
```

The `-seg.pt` checkpoint predicts object masks as well as classes and boxes. A mask identifies pixels assigned to an object instance. The saved result contains four people, one bus and one stop sign. The detection and segmentation networks are separate models, so their retained objects can differ.

The saved mask tensor has shape (6, 640, 480): six instances, each represented on a grid of 640 rows and 480 columns. This resolution differs from the original photo's (1080, 810) spatial resolution. Model preprocessing and mask generation determine the mask grid.

If no masks are predicted, `r.masks` can be `None`. Check for that before reading its shape in code intended for arbitrary photos. The boundaries are model estimates and can include small errors, missing regions or background pixels.

17.3 Using the bus mask to make a cutout

```
# Use a mask: cut the bus out of the photo
names = [r.names[int(c)] for c in r.bboxes.cls]
bus_mask = r.masks.data[names.index("bus")].cpu().numpy()
bus_mask = cv2.resize(bus_mask, (rgb.shape[1], rgb.shape[0])) > 0.5
cut = np.where(bus_mask[..., None], rgb, 255)
show(cut, "Only the bus pixels")
```

The list `names` follows the same object order as the masks. `names.index("bus")` finds the first bus. It raises an error if no bus is present, and selects only the first if there are several.

`cpu()` moves the tensor to CPU memory and `numpy()` gives NumPy access to its values. The `resize` arguments use the original width and height. The comparison `> 0.5` makes a Boolean mask after interpolation. `bus_mask[..., None]` adds a final one-element channel dimension, allowing the same mask decision to apply to red, green and blue. `np.where` keeps the original pixel where the mask is true and substitutes 255 in every channel elsewhere, producing white.

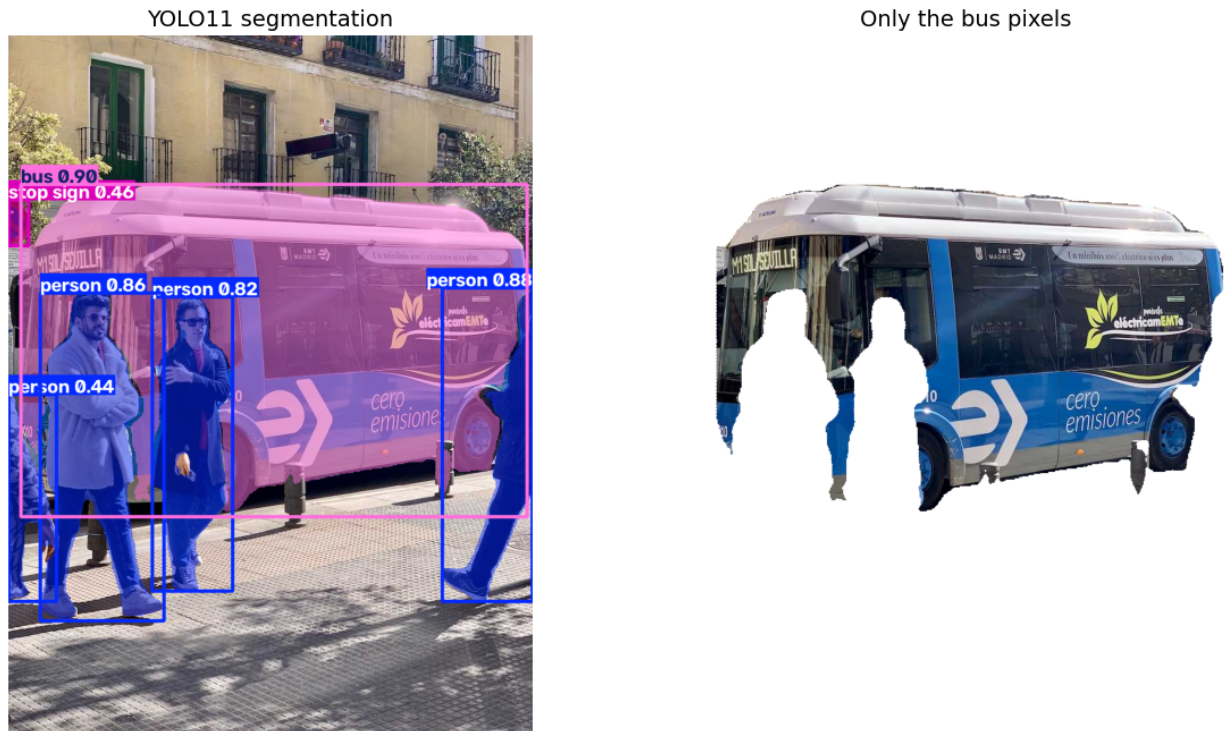


Figure 21: Saved instance segmentation and the bus cutout made from its mask.

For this image, the saved mask and original image share the same aspect ratio. General image pipelines can add padding during resizing. Stretching a padded mask directly over the original photo can misalign it. One convenient alternative is to use the result's mask polygons, whose `xy` coordinates are expressed in the original image coordinate system:

```
result = seg_model("bus.jpg", verbose=False)[0]
mask = np.zeros(rgb.shape[:2], dtype=np.uint8)
if result.masks is not None:
    for i, class_id in enumerate(result.boxes.cls):
        if result.names[int(class_id)] == "bus":
            polygon = np.round(result.masks.xy[i]).astype(np.int32)
            if len(polygon) >= 3:
                cv2.fillPoly(mask, [polygon], 1)
cut = np.where(mask[..., None].astype(bool), rgb, 255)
show(cut, "Predicted bus regions")
```

This supplementary example combines predicted bus polygons. An image with no bus produces a white output. Polygon rasterisation is an approximation to the model mask and can simplify boundaries. Keep the image array and prediction result associated with the same input image.

17.4 Pose estimation

```
# Pose estimation: 17 body key points per person
pose_model = YOLO("yolo11n-pose.pt")
r = pose_model("zidane.jpg", verbose=False)[0]
print("People found:", len(r.keypoints))
print("Key points per person:", r.keypoints.xy.shape[1])
show(r.plot()[ :, :, ::-1], "YOLO11 pose", size=(12, 7))
```

The `-pose.pt` model predicts human body keypoints. The saved photograph has two detected people and 17 keypoints per person. `r.keypoints.xy` has shape $(N, 17, 2)$, where N is the number of people and the final two values are x and y coordinates in the image.

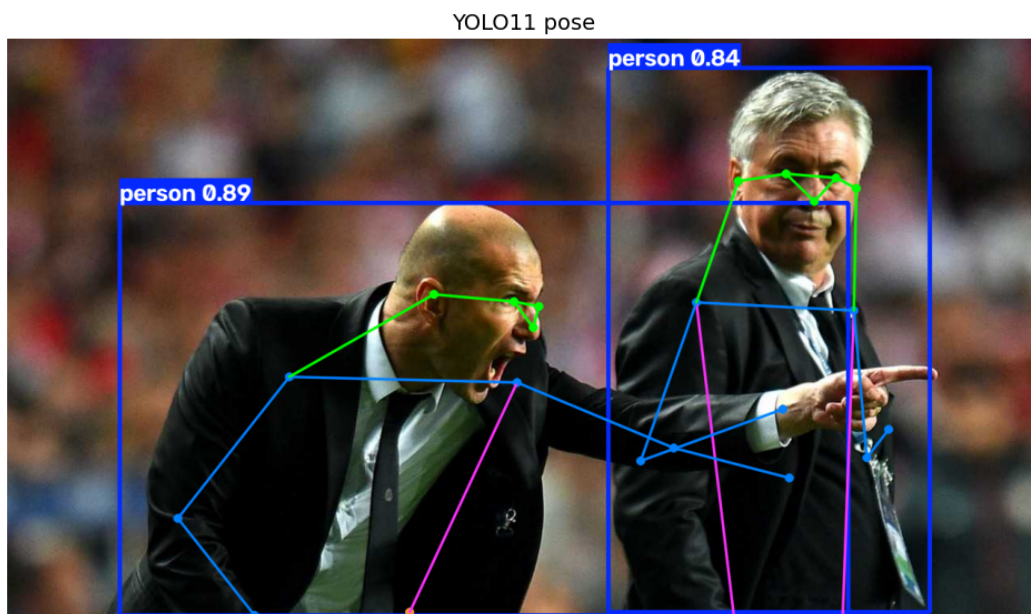


Figure 22: The saved pose output. Lines join predicted anatomical keypoints according to a fixed skeleton layout.

```
# The 17 COCO key points, in order
kp_names = ["nose", "left eye", "right eye", "left ear", "right ear", "left shoulder", "right shoulder",
            "left elbow", "right elbow", "left wrist", "right wrist", "left hip", "right hip",
            "left knee", "right knee", "left ankle", "right ankle"]
person = r.keypoints.xy[0].cpu().numpy()
for name, (x, y) in zip(kp_names, person):
    print(f"{name:15s} x={x:6.0f} y={y:6.0f}" + (" (not visible)" if x == 0 and y == 0 else ""))
```

The list gives the 17 keypoint names in their required order. `[0]` selects the first detected person. The loop pairs each name with its coordinate pair, and `:6.0f` prints a six-character field with no decimal places. “Left” and “right” refer to the person’s anatomical sides, which need not match the viewer’s sides.

The code appends “not visible” when both coordinates are zero. This is a simple display convention, not a reliable visibility assessment for every version or input. Models can estimate positions for obscured joints, and a nonzero coordinate does not establish that the joint is visible or accurately located. Examine keypoint confidence, available through `r.keypoints.conf` when supplied, and

inspect the image. Also check that at least one person exists before indexing person zero.

These coordinates describe a two-dimensional image. They do not directly provide a person's three-dimensional joint positions or physical distances. Perspective and camera placement change the apparent geometry.

17.5 How keypoints can support an application

For an elbow-angle estimate, let S , E and W be shoulder, elbow and wrist coordinates. Form vectors $u = S - E$ and $v = W - E$. Their image-plane angle is

$$\theta = \cos^{-1} \left(\frac{u \cdot v}{\|u\| \|v\|} \right).$$

Use sufficiently confident points, avoid zero-length vectors, and clip the computed cosine into $[-1, 1]$ to handle rounding. Counting repetitions then requires a sequence of observations and rules for movement phases. A dependable fall-detection application requires much more evidence and validation than one frame's pose. Pose keypoints also do not establish a person's identity.

18 Your own photographs, video and homework

18.1 Understanding the upload cell

```
# Upload your own image (works in Colab). Outside Colab, set my_file to any image path.
my_file = "bus.jpg"
try:
    from google.colab import files
    uploaded = files.upload()
    my_file = list(uploaded.keys())[0]
except Exception as e:
    print("Not in Colab (or nothing uploaded) - using bus.jpg instead")

for m, title in [(model, "Detection"), (seg_model, "Segmentation"), (pose_model, "Pose")]:
    res = m(my_file, verbose=False)[0]
    show(res.plot()[:, :, :-1], f"{title}: {my_file}", size=(9, 9))
```

The default input is `bus.jpg`. In Colab, `files.upload()` opens a file chooser and returns a mapping containing uploaded filenames. The code takes the first filename. Outside Colab, set `my_file` to a valid local image path.

The broad `except Exception` catches any error in the upload block, including an empty upload and import errors, then retains the bus image. Its printed message is therefore a fallback description, not a precise diagnosis of the error. The saved outputs in the uploaded notebook use that fallback example.

The loop pairs each loaded model with a title, runs inference and displays the annotated result. It reuses the existing detection, segmentation and pose models. Uploading a photo for prediction does not train those models or teach them a new class.

A detector trained on COCO can return only names from its detection label set. An unfamiliar object may receive no detection, or may be confused with an available category. It is not guaranteed to choose a nearest label for every visible object. The pose checkpoint predicts human keypoints; its output is not an 80-category object list.

To learn a custom task, collect representative labelled examples and separate training, validation and test data. Classification uses image-level labels; detection needs boxes and classes; instance

segmentation needs masks; pose needs keypoint annotations and conventions. The required annotations follow the output you want the model to learn.

18.2 Video and a local webcam

```
model("my_video.mp4", save=True)
model(source=0, show=True)
```

A video is a sequence of frames. The first command predicts on a video file and saves an annotated output under the run directory. The second requests camera device zero and a display window. It requires access to a local camera and a suitable graphical environment; a remote Colab runtime does not obtain the laptop webcam through this command.

Frame-by-frame detection does not automatically maintain a stable identity for each object. Tracking adds associations across frames. Counting all detections across a video would repeatedly count the same visible person. A crossing counter needs tracking or another explicit temporal counting method.

Hardware deployment also involves camera input, supported model formats, memory, power and measured latency. A model suitable for a laptop is not automatically suitable for every Raspberry Pi or microcontroller.

18.3 A careful three-photo exercise

Choose three different photos and record the image description, model checkpoint and confidence threshold. For each image, list retained classes and counts, inspect missed objects and incorrect labels, and compare detection, segmentation and pose outputs where relevant.

Change one setting at a time. For example, compare thresholds 0.25 and 0.6 on the same image and checkpoint. Explain which predictions were removed and whether they were correct. Three photos make a useful qualitative exercise; a performance estimate needs a larger representative labelled evaluation set.

For the CNN homework, add the third convolution using the earlier solution, verify the shape and parameter count, and compare validation curves under the same training conditions. Record a measured result with its settings. Keep the test set for final evaluation after choosing the approach.

19 Connecting vision to other deep learning ideas

19.1 Sequential data, RNNs and LSTMs

A sequence has an order that carries information: words in a sentence, sound samples, sensor readings or video frames. A recurrent neural network processes steps using a hidden state, a vector summarising information carried from earlier steps. A simple update is

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

Here x_t is the current input, h_{t-1} the previous state, and h_t the updated state. The same weight matrices are reused over time. The state has limited capacity, and learning relationships across many steps can be difficult when gradients become very small or very large.

An LSTM, or long short-term memory network, introduces a cell state and learned gates. A forget gate controls retained information, an input gate controls new information, and an output gate

controls what becomes available through the hidden state. A gate typically uses sigmoid values between zero and one to scale information element by element. These mechanisms help the network learn longer dependencies, although success still depends on data and training.

19.2 Attention and Transformers

Attention allows a representation to combine information from other positions with learned weights. A word such as “it” may need information from an earlier noun. Attention computes compatibility scores between representations and uses normalised scores to form a weighted combination.

In scaled dot-product attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Queries Q describe what each position seeks; keys K describe what positions offer for matching; values V carry the information to combine. These are learned linear transformations of representations. Dot products measure compatibility, d_k is the key-vector dimension, and softmax converts each row of scores into nonnegative weights summing to one. Multiplying by V forms the weighted combinations.

Transformers combine attention with feed-forward layers, residual connections and normalisation. Positional information describes token order. A causal mask prevents a next-token predictor from attending to future tokens during training. Many positions can be processed in parallel during training, while ordinary autoregressive generation produces new tokens successively.

19.3 Vision Transformers

A Vision Transformer can divide an image into patches and convert each patch into a token representation. For a 224×224 RGB image with 16×16 patches, there are

$$(224/16)^2 = 14^2 = 196 \text{ patches.}$$

Each patch initially contains $16 \times 16 \times 3 = 768$ colour values. A learned projection maps those values to an embedding vector. Positional information identifies patch locations, attention combines information across patches, and an output head makes the prediction. The embedding dimension is a design choice and need not equal 768.

These topics are an overview from the slides. The supplied notebooks train dense and convolutional networks and use YOLO models; they do not implement RNN, LSTM or Transformer training.

19.4 Application ideas from the slides

Agricultural image classification can assign a crop-leaf image to a labelled condition, while detection can locate several leaves or fruit. Health-image tasks can involve classification, localisation or segmentation according to the intended output. Traffic applications may combine vehicle detection with a separate number-plate detector and text-recognition model. Shelf monitoring can detect products and then apply inventory rules.

Each application needs labels and evaluation appropriate to its environment. A general COCO detector does not already know every crop disease, medical finding, product identity or person’s name. Recognition of identity and attendance also require additional components beyond person detection. Camera quality, illumination, occlusion and differences between training and deployment images affect results.

The next day's topics connect learning systems to devices, robotics and other applications, including ESP32 and Raspberry Pi examples, biomedical uses, reinforcement learning and generative AI. Today's preparation is understanding the model's input, output, learned parameters, evaluation and resource requirements.

20 Review questions with explained answers

Pause and try. Why divide Fashion-MNIST pixel values by 255?

The stored intensities range from 0 to 255. Dividing by 255 maps them into $[0, 1]$, giving the network consistently scaled inputs. This operation changes the numerical representation while retaining the relative intensity pattern.

Pause and try. How many weights and biases connect 784 inputs to 128 dense units?

Each unit receives 784 weights and one bias, so the layer has $(784 + 1)128 = 100,480$ parameters. The final ten-class layer adds $(128 + 1)10 = 1,290$, making 101,770 overall.

Pause and try. Why is a loss needed if we already calculate accuracy?

Training needs a quantity that measures how its numerical predictions should change. Cross-entropy reacts to the probability assigned to the true class and provides gradients for optimisation. Accuracy records whether the selected class is correct and can remain unchanged while probabilities improve.

Pause and try. What does dropout rate 0.3 mean?

During training, each affected activation is independently set to zero with probability 0.3 under the usual dropout operation. Retained values are scaled by $1/0.7$ to preserve their expected contribution. Ordinary inference uses the full layer without this random dropping.

Pause and try. What is the output of a valid three-by-three convolution on a 28-by-28 image with 32 filters?

With stride one, each spatial dimension becomes $28 - 3 + 1 = 26$. The output is $26 \times 26 \times 32$. Each filter supplies one output channel. Input channels affect the filter's depth and parameter count.

Pause and try. Why can pooling reduce the following dense layer's parameter count?

Pooling reduces spatial dimensions. Flattening a smaller feature map yields fewer inputs, and a dense layer has one weight per input-output connection. Ordinary max pooling itself has no learned weights.

Pause and try. What can we conclude from the transfer experiment?

In the stored two-epoch run on this closely related dataset, pretrained starting weights produced higher validation accuracy. The experiment does not measure equal total lifetime training compute, long-run limits, or performance on every new domain.

Pause and try. Does a detector's count equal the actual number of objects?

It counts retained predictions. Missed objects reduce that count, while false positives or duplicate detections can increase it. Evaluate counts against labelled examples and a clear definition of what should be counted.

Pause and try. Why might a pose coordinate be misleading?

A joint may be obscured, outside the frame or estimated with low confidence. A coordinate can still be returned. Inspect confidence and visual evidence before using it, and remember that image coordinates describe a projection of the body.

21 A compact vocabulary reference

Term	Meaning in this lesson
Tensor	An array with one or more dimensions, such as a batch of images.
Parameter	A learned number, commonly a weight or bias.
Hyperparameter	A chosen setting such as layer width, learning rate or batch size.
Activation	A layer's output value; also the function used to transform its weighted input.
Logit	A raw class score before probability normalisation.
Loss	A numerical training objective measuring prediction error.
Gradient	Derivatives describing how a small parameter change affects the loss.
Backpropagation	Computing parameter gradients by applying the chain rule through the network.
Optimiser	The rule that uses gradients and possibly accumulated statistics to update parameters.
Batch	A group of examples processed together for a training update.
Epoch	One pass through the selected training examples.
Validation set	Held-out examples used to monitor development and select settings.
Test set	Held-out examples reserved for final performance assessment.
Regularisation	A method intended to improve generalisation by constraining or modifying learning.
Feature map	Spatial responses produced by a convolutional filter or layer.
Kernel	The small array of coefficients used in a convolution or filtering operation.
Padding	Added border values used to control spatial processing and output dimensions.
Stride	The step between successive filter positions.
Checkpoint	Saved model state, often including weights and training metadata.
Fine-tuning	Updating a model that starts from previously learned weights.
Inference	Applying a trained model to an input to obtain a prediction.
Bounding box	A rectangle estimating an object's location.
Mask	An array indicating membership in a predicted region.
Keypoint	A named landmark such as a wrist or shoulder.
Augmentation	Transformations applied to training examples with corresponding label handling.

22 Sources, coverage and result provenance

These notes follow the supplied Day 4 slide deck, speaker notes and all three notebooks. Every nonempty code cell is reproduced in the relevant explanation, with typography in code comments simplified where necessary for LaTeX. The exercises are placed beside the concepts they practise. Selected saved notebook figures illustrate the explanations.

The numerical model results and example predictions are taken from the uploaded notebook outputs or explicitly attributed to the slides and speaker notes. The models were not retrained when preparing this document. Parameter counts, tensor dimensions and worked arithmetic were checked independently. The optional exercise cells without stored outputs are explained as proposed runs.

For results that differ across the supplied materials, the notes identify the source of each value. The dense-network comparison in Notebook 2 contains hard-coded earlier scores; those entries do not constitute a new evaluation of the models trained in Notebook 1.

22.1 Primary references for further reading

The following references support the API behaviour and concepts used here. Package behaviour can evolve; compare documentation with the version installed in your notebook.

1. Fashion-MNIST dataset, labels and file format: <https://github.com/zalandoresearch/fashion-mnist>.
2. Keras training and evaluation APIs, including validation splitting: https://keras.io/api/models/model_training_apis/.
3. Keras convolution layers: https://keras.io/api/layers/convolution_layers/convolution2d/.
4. Keras dropout behaviour: https://keras.io/api/layers/regularization_layers/dropout/.
5. Keras early stopping: https://keras.io/api/callbacks/early_stopping/.
6. OpenCV image filtering: https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html.
7. Ultralytics YOLO11 task variants: <https://docs.ultralytics.com/models/yolo11/>.
8. Ultralytics prediction usage: <https://docs.ultralytics.com/modes/predict/>.
9. Ultralytics results, boxes, masks and keypoints: <https://docs.ultralytics.com/reference/engine/results/>.
10. Imagenette dataset documentation: <https://docs.ultralytics.com/datasets/classify/imagenette/>.
11. Vaswani and colleagues, *Attention Is All You Need*: <https://arxiv.org/abs/1706.03762>.
12. Dosovitskiy and colleagues, *An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale*: <https://arxiv.org/abs/2010.11929>.